S.I.G.S.M

Administración de Sistemas Operativos

Empre S.A.S.

Rol	Apellido	Nombre	C.I	Email
Coordinador	Cabrera	Martin	4919741-2	empre.sa.utu.isbo@gmail.com
Subcoordinador	Sena	Angel	5341149-0	jeansangel2003@gmail.com

Docente: Martinez, Santiago

Fecha de culminación

09/11/2026

TERCERA ENTREGA

Grupo: Empre S.A.S.

Fecha: 09/11/2026

Empre S.A.S.

Montevideo, 02 de junio de 2026

Profesor Santiago Martinez
Asignatura: Administración de Sistemas Operativos
Instituto Superior Brazo Oriental

Presente.
A continuación, los alumnos de tercero 3MJ del turno vespertino del Instituto Superior Brazo Oriental se presentan ante usted, con el fin de informar la creación del grupo
Empre S.A.S. Los correspondientes integrantes con su roles son los siguientes:

A continuación, se detalla dicha integración y roles del grupo:

ROL	C.I	APELLIDO	NOMBRE	E-MAIL
Coordinador	4919741-2	Cabrera	Martin	mdcabrera@hotmail.es
Subcoordinador	5341149-0	Sena	Angel	jeansangel2003@gmail.com

Por contacto al correo: empre.sa.utu.isbo@gmail.com

Firma:





COORDINADOR

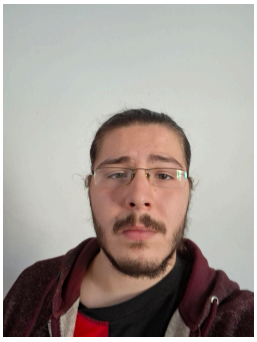
SUBCOORDINADOR

I.S.B.O

3MJ

Grupo: Empre S.A.S.

Fecha: 09/11/2026



Cabrera, Martin

COORDINADOR



Sena, Angel

SUBCOORDINADOR

I.S.B.O

3MJ

1. Introducción..... 9

2. Requisitos previos..... 9

2.1 Sistema operativo elegido y soporte técnico..... 10

2.2 Software necesario..... 11

2.3 Requisitos mínimos de la máquina virtual..... 11

2.4 Hardware del equipo anfitrión..... 12

2.5 Requisitos de un PC cliente..... 13

2.6 Tablet administrativa..... 13

2.7 Notebook administrativa..... 14

2.8 PC administrativo..... 15

2.9 Requisitos básicos para consulta de documentación..... 15

3. Costos y formas de obtener el servidor..... 16

3.1 Servidor propio..... 17

3.2 Servidor contratado..... 18

3.3 Máquina virtual..... 19

3.4 Bare metal..... 21

3.5 Opción seleccionada para el proyecto..... 22

4. Creación de la máquina virtual..... 23

5. Instalación de Fedora Server..... 26

6. Actualización inicial del sistema y política de actualizaciones..... 30

7. Configuración de red inicial..... 32

8. Configuración básica de SSH..... 33

9. Configuración básica del firewall..... 34

10. Instalación de XAMPP..... 35

11. Inicio y verificación de XAMPP..... 37

12. Verificación de Apache, PHP y MariaDB..... 38

12.1 Verificación de Apache..... 38

12.2 Verificación de PHP..... 39

12.3 Verificación de MariaDB..... 40

13. Instalación y configuración de Git..... 41

14. Pruebas y evidencias de la instalación..... 42

14. Consideraciones de seguridad iniciales..... 47

15. Vulnerabilidades e incidentes de seguridad pasados..... 49

15.1 Apache HTTP Server..... 49

15.2 PHP..... 50

15.3 MariaDB..... 51

15.4 OpenSSH..... 52

15.5 Git..... 53

16. Configuración de red y acceso remoto al servidor..... 54

16.1 Introducción..... 54

16.2 Configuración de dirección IP estática..... 55

16.2.1 Identificación de la interfaz de red..... 56

16.2.2 Configuración mediante nmtui..... 56

16.2.3 Aplicación de la configuración..... 58

16.2.4 Verificación de conectividad..... 59

16.3 Configuración de SSH mediante clave pública y privada..... 60

16.3.1 Generación del par de claves..... 61

16.3.2 Incorporación de la clave pública al servidor..... 62

16.3.3 Prueba de conexión mediante clave.....63
16.3.4 Desactivación de autenticación mediante contraseña.....64
16.3.5 Resultados de la configuración.....65
17. Configuración de grupos y permisos..... 65
17.1 Introducción..... 65
17.2 Grupos administrativos definidos..... 66
17.2.1 Administrador del firewall..... 66
17.2.2 Administrador de Base de Datos..... 66
17.3 Configuración de permisos mediante sudoers..... 67
17.3.1 Acceso seguro al archivo..... 67
17.3.2 Definición de comandos permitidos..... 69
17.3.3 Aplicación de permisos a los grupos..... 70
18. Docker..... 71
18.1 Configuración del repositorio de Docker..... 71
18.2 Instalación de Docker Engine..... 72
18.3 Inicio y habilitación del servicio..... 73
18.4 Comprobación de la instalación..... 73
19. Política y gestión de respaldos..... 74
19.1 Introducción..... 74
19.2 Información incluida en los respaldos..... 75
19.2.1 Respaldo de la base de datos..... 75
19.2.2 Respaldo de documentos de la aplicación..... 76
19.3 Política de respaldos..... 77
19.3.1 Respaldo local diario..... 78

19.3.2 Respaldo remoto semanal..... 78
19.4 Ubicación de los respaldos..... 79
19.4.1 Almacenamiento local..... 79
19.4.2 Almacenamiento remoto..... 80
20. Dockerización y despliegue de S.I.G.S.M..... 81
20.1. Introducción a la dockerización..... 81
20.2. Justificación de la utilización de Docker..... 82
20.2.1. Transición del entorno XAMPP a contenedores..... 82
20.2.2. Ventajas de Docker para el despliegue del sistema..... 83
20.2.3. Utilización de Docker Compose..... 85
20.3. Arquitectura de contenedores..... 86
20.3.1. Organización general de los servicios..... 86
20.3.2. Frontend: React y Nginx..... 87
20.3.3. Backend: PHP y Apache..... 88
20.3.4. Base de datos: MariaDB..... 89
20.3.5. Servicios auxiliares: admin-init y seed-demo..... 90
20.3.6. Comunicación y red interna entre contenedores..... 91
20.4. Estructura y archivos de configuración..... 93
20.4.1. Organización del directorio Docker..... 93
20.4.2. Construcción de imágenes mediante Dockerfiles..... 94
20.4.3. Configuración del archivo compose.yaml..... 97
20.4.4. Configuración de Apache y Nginx..... 98
20.4.5. Variables de entorno y archivo .env..... 101
20.5. Despliegue de la aplicación..... 102
20.5.1. Preparación del servidor..... 103
20.5.2. Obtención de los archivos desde GitHub..... 104

20.5.3. Configuración inicial del entorno..... 105
20.5.4. Construcción e inicio de los contenedores..... 106
20.5.5. Inicialización de la base de datos..... 108
20.5.6. Creación del administrador inicial..... 109
20.5.7. Carga opcional de datos de demostración..... 110
20.5.8. Verificación del despliegue..... 112
20.6. Persistencia y respaldos..... 114
20.6.1. Volúmenes utilizados por la aplicación..... 115
20.6.2. Persistencia de MariaDB y documentos..... 116
20.6.3. Adaptación de los scripts de respaldos a Docker..... 117
20.6.4. Automatización de respaldos mediante cron..... 120
20.7. Administración y mantenimiento de los contenedores..... 124
20.7.1. Inicio, detención y reinicio de servicios..... 125
20.7.2. Consulta de estados y registros..... 126
20.7.3. Actualización de la aplicación desde GitHub..... 129
20.8. Seguridad del entorno Docker..... 132
20.8.1. Aislamiento de servicios y exposición de puertos..... 133
20.8.2. Gestión de credenciales..... 134
20.8.3. Permisos de administración de Docker..... 135
20.8.4. Consideraciones para un despliegue en producción..... 136
20.9. Conclusiones..... 137
21. Bibliografía..... 139

Manual de instalación del servidor

1. Introducción

El presente documento describe el proceso de instalación y configuración de un servidor destinado al entorno de pruebas y eventual despliegue del sistema, así como el hardware y software necesarios para llevarlo a cabo. Este servidor permitirá alojar los servicios necesarios para el funcionamiento de los dos módulos que se solicitan para el proyecto.

El objetivo de este manual es documentar un procedimiento claro, ordenado y verificable para preparar un entorno compatible con los requerimientos técnicos del proyecto. A través de esta instalación se busca disponer de una base funcional sobre la cual desarrollar, probar y validar los módulos del sistema antes de su despliegue definitivo.

La configuración propuesta contempla la instalación de los componentes principales necesarios para ejecutar una aplicación web institucional, priorizando la estabilidad, la organización del entorno, la seguridad básica del servidor y la posibilidad de administrar el sistema de forma remota.

2. Requisitos previos

Para la instalación del servidor se utilizará Fedora Server 43 como sistema operativo. Esta distribución permite preparar un entorno de servidor estable, actualizado y adecuado para instalar servicios web, herramientas de administración, acceso remoto y componentes necesarios para el despliegue de aplicaciones. Por

ende, necesitaremos una imagen del sistema operativo, la cual se puede encontrar en el sitio oficial de Fedora Server: https://fedoraproject.org/server/download/.

El servidor se implementará inicialmente sobre una máquina virtual, lo que permite trabajar en un entorno controlado, fácil de replicar y seguro para realizar pruebas sin afectar equipos físicos. Esta modalidad facilita la documentación del proceso de instalación, la toma de evidencias y la corrección de configuraciones durante el desarrollo del proyecto.

2.1 Sistema operativo elegido y soporte técnico

El sistema operativo seleccionado para el servidor será Fedora Server 43. La elección de esta distribución se fundamenta en que permite trabajar sobre un entorno GNU/Linux orientado a servidores y compatible con los servicios requeridos por el sistema, además de incorporar de forma nativa varios de los paquetes que se utilizarán durante el desarrollo del proyecto y su eventual despliegue.

Fedora Server resulta adecuado para el entorno de pruebas del proyecto porque cuenta con un sistema de gestión de paquetes mediante dnf, el cual permite instalar y administrar servicios de forma clara desde la propia terminal, también ofrece una base moderna para configurar aplicaciones web institucionales. Además su uso en una máquina virtual permite simular el entorno de servidor real sin necesidad de adquirir hardware dedicado.

En cuanto al soporte técnico, Fedora cuenta principalmente con soporte comunitario, documentación oficial, foros, reportes de errores, repositorios actualizados y avisos de seguridad. Este tipo de soporte resulta suficiente para un

entorno académico de pruebas permitiéndonos consultar documentación técnica, resolver incidencias comunes y mantener el sistema actualizado. (Boy et al., 2026)

Sin embargo, para un entorno de producción definitivo en una institución como el Hospital de Clínicas se podría evaluar el uso de una distribución con soporte empresarial, como Red Hat Enterprise Linux. Esta alternativa ofrece contratos de soporte formal, mantenimiento extendido, asistencia técnica oficial y mayor estabilidad a largo plazo. Por este motivo se deberá analizar junto al Departamento Técnico de Informática la distribución más adecuada según sus políticas internas de soporte y mantenimiento.

2.2 Software necesario	
Herramienta	**Uso**
Fedora Server 43	Sistema operativo del servidor
Oracle VirtualBox 7.2.4	Creación y administración de la máquina virtual
XAMPP para linux	Paquete integrado con Apache, MariaDB y PHP

2.3 Requisitos mínimos de la máquina virtual	
Recurso	**Valor recomendado**
Procesador	2 núcleos
Memoria RAM	4 GB
Disco Duro	50 GB
Red	Adaptador puente o NAT

2.3 Requisitos mínimos de la máquina virtual	
Sistema Operativo	Fedora Server 43

Nota. (Boy et al., 2026)

Estos valores permiten ejecutar correctamente un entorno inicial compuesto por un servidor web, PHP, bases de datos y herramientas de administración del sistema. Para un entorno de producción ha de tenerse en cuenta la cantidad de usuarios, el volumen de documentos, la cantidad de registros y el uso de la base de datos, entre otros, para escalar de manera adecuada los recursos del servidor de ser necesario.

2.4 Hardware del equipo anfitrión

Como el servidor será implementado mediante una máquina virtual, es necesario considerar también los recursos del equipo anfitrión. Este deberá contar con capacidad suficiente para ejecutar simultáneamente el sistema operativo principal, el software de virtualización y Fedora Server 43. Se recomiendan los siguientes componentes mínimos para el correcto funcionamiento: procesador de al menos cuatro núcleos, 8GB de RAM, y por lo menos 60GB de almacenamiento mediante SSD lo que mejorará el rendimiento del sistema. Si bien estos son los requerimientos mínimos recomendados, a continuación se detalla el hardware que utilizará el PC anfitrión durante el proyecto.

Recurso	Valor
Procesador	AMD Ryzen 5 5600X 6-Core
Memoria RAM	32 GB DDR4
Almacenamiento	1 TB SSD

Tarjeta de red	Realtek PCIe GbE Controller
Sistema Operativo	Windows 11

2.5 Requisitos de un PC cliente

Los requisitos de un PC cliente varían según el tipo de usuario y módulo del sistema al que se desea acceder. Por este motivo se separan los requisitos según el perfil que utilizará la aplicación.

Las secciones administrativas del sistema están pensadas para ser utilizadas por funcionarios que gestionan información desde un navegador web. A diferencia del paciente que accede mediante un código QR desde el teléfono, los funcionarios requieren un equipo más cómodo para trabajar con formularios, carga de archivos, filtros de búsqueda y seguimiento de registros.
Por este motivo se establecen los requisitos mínimos y recomendados para tres tipos de dispositivos: tablet, notebook, y PC de escritorio.

2.6 Tablet administrativa		
Recurso	**Requisito mínimo**	**Requisito recomendado**
Procesador	4 núcleos	8 núcleos
Memoria RAM	4GB	6GB o más
Almacenamiento libre	8GB	16GB
Pantalla	10 pulgadas	10 a 12 pulgadas
Resolución	1280 x 800	1920 x 1080
Conectividad	Wi-Fi	Wi-Fi

2.6 Tablet administrativa		
Navegador	Chrome, Firefox, Edge actualizado	Chrome, Firefox, Edge actualizado
Periféricos	No obligatorio	Teclado externo recomendado

2.7 Notebook administrativa		
Recurso	**Requisito mínimo**	**Requisito recomendado**
Procesador	2 núcleos	4 núcleos o más
Memoria RAM	4GB	8GB o más
Almacenamiento libre	50 GB	100 GB o más
Pantalla	13 pulgadas	14 pulgadas o más
Resolución	1366 x 768	1920 x 1080
Conectividad	Wi-Fi estable	Wi-Fi / Ethernet
Navegador	Chrome, Firefox, Edge actualizado	Chrome, Firefox, Edge actualizado
Periféricos	Touchpad y teclado integrados	Mouse externo recomendado
Sistema Operativo	Windows, GNU/Linux, macOS actualizado	Windows, GNU/Linux, macOS actualizado

2.8 PC administrativo		
Recurso	**Requisito mínimo**	**Requisito recomendado**
Procesador	2 núcleos	4 núcleos o más
Memoria RAM	4GB	8GB o más
Almacenamiento libre	50 GB	100 GB o más
Pantalla	19 pulgadas	22 pulgadas o más
Resolución	1366 x 768	1920 x 1080
Conectividad	Ethernet cableado	Ethernet cableado
Navegador	Chrome, Firefox, Edge actualizado	Chrome, Firefox, Edge actualizado
Periféricos	Teclado y mouse	Teclado y mouse
Sistema Operativo	Windows, GNU/Linux, macOS actualizado	Windows, GNU/Linux, macOS actualizado

Los equipos destinados a las secciones administrativas no requieren prestaciones elevadas ya que el sistema funcionará mediante navegador web y la mayor parte del procesamiento la realizará el servidor. Sin embargo, sí deben contar con recursos suficientes para garantizar una navegación fluida, la correcta visualización de los formularios y la carga de documentos.

2.9 Requisitos básicos para consulta de documentación

Para los pacientes que accedan a la documentación mediante código QR los requisitos del equipo serán mínimos. El acceso está pensado principalmente para teléfonos inteligentes, por lo que no será necesario instalar software adicional ni utilizar un PC.

Como requisito mínimo el dispositivo deberá contar con una cámara funcional con la cual poder escanear el código QR, un navegador actualizado, conexión a internet estable y memoria suficiente para cargar correctamente el contenido. Se consideran los siguientes recursos: un teléfono con procesador de al menos 2 núcleos, 2GB de RAM, pantalla de 5 pulgadas o superior y 1GB de almacenamiento libre.

De forma recomendada se sugiere utilizar un dispositivo con 4 núcleos, 3 GB de RAM o más, pantalla de 6 pulgadas, conexión 4/5G o Wi-Fi estable y un sistema operativo actualizado (ya sea Android o iOS). Estas características permiten una experiencia fluida al visualizar documentos, acceder a formularios o navegar el contenido publicado por el hospital.
En este perfil de usuario la prioridad está en garantizar el acceso simple, claro y compatible a la documentación pertinente.

3. Costos y formas de obtener el servidor

Para implementar el servidor del sistema se analizaron distintas alternativas. La elección del tipo de servidor influye directamente en el costo inicial, el mantenimiento, la seguridad, disponibilidad y la facilidad de administración.

En el análisis se consideran cuatro opciones principales: servidor propio, servidor contratado, máquina virtual y servidor bare metal. Cada alternativa presenta ventajas y limitaciones, por lo que resulta necesario compararlas antes de seleccionar la opción más adecuada para el entorno del proyecto.

3.1 Servidor propio

Un servidor propio consiste en adquirir o reutilizar un equipo físico destinado a alojar los servicios del sistema. En este caso el hardware queda bajo control directo de la organización permitiendo a esta administrar el sistema operativo, la red, discos, copias de seguridad y políticas de seguridad de forma interna. (Smith, 2026)

Esta alternativa ofrece un gran control sobre la infraestructura ya que los datos permanecen alojados en equipos administrados por la propia institución. Además permite adaptar el servidor según las necesidades del proyecto, ampliando memoria, almacenamiento o capacidad de procesamiento de ser necesario. (Smith, 2026)

Sin embargo esta alternativa implica mayores responsabilidades dado que se debe considerar el costo de adquisición del equipo, el consumo eléctrico, el mantenimiento del dispositivo, la refrigeración, la protección eléctrica y los respaldos. Además ante una falla de hardware la recuperación puede depender de la disponibilidad de repuestos o del equipo técnico correspondiente. (Smith, 2026)
Para el proyecto planteado un servidor propio podría ser adecuado a la hora de encarar un entorno definitivo de producción, especialmente si se pretende mantener la información dentro de la infraestructura interna del hospital. No obstante, para la etapa inicial del proyecto puede representar un costo innecesario si se dispone de antemano de equipos capaces de ejecutar una máquina virtual.

Concepto	Costo aproximado
Equipo físico básico para servidor	Medio / Alto
Sistema Operativo Fedora Server	Sin costo de licencia
Mantenimiento físico	Variable
Consume eléctrico	Permanente
Repuestos o ampliaciones	Variable
Administración técnica	Interna

3.2 Servidor contratado

Un servidor contratado consiste en utilizar infraestructura provista por un tercero, generalmente mediante servicios de hosting, VPS o cloud. En este modelo la empresa proveedora se encarga del hardware físico, la energía, conectividad, disponibilidad del centro de datos y, dependiendo del servicio contratado, parte del mantenimiento de la plataforma. (Smith, 2026)

Esta alternativa reduce la necesidad de contar con hardware propio y permite obtener un servidor funcional en poco tiempo. Además facilita la ampliación de recursos dado que muchos proveedores permiten aumentar memoria, almacenamiento o poder de procesamiento según sea requerido.

Dentro de este tipo de soluciones existen distintas opciones, como servidores VPS o servicios cloud. Para este análisis se toma como referencia **Amazon Web Services (AWS)** por ser uno de los proveedores cloud más utilizados y por permitir estimar costos de forma pública a partir de sus planes.

En AWS una de las opciones más simples para contratar un servidor es Amazon Lightsail. Este servicio ofrece servidores con recursos predefinidos, precio mensual fijo e incluye CPU, memoria RAM, almacenamiento SSD y transferencia de datos dentro del mismo plan, esto representa una alternativa viable, especialmente si se busca que el sistema esté disponible desde Internet sin depender de una red local o de un equipo físico propio. También permite escalar recursos con facilidad si aumentan los documentos, encuestas o registros de traslados. (Amazon AWS, n.d.)

Sin embargo, para este proyecto no se selecciona como primera opción ya que implica un coste mensual fijo y requiere evaluar cuidadosamente aspectos como privacidad, ubicación de los datos, control de accesos, respaldos y dependencia del proveedor. Por este motivo el servidor contratado se considera una alternativa válida para comparar costos o para una posible implementación futura, mientras que para la etapa de pruebas se prioriza el uso de una máquina virtual local.

Opción	Recursos	Costo
Lightsail 2 GB	1 vCPU, 2 GB RAM, 60GB SSD, 3 TB transferencia	12 USD/mes
Lightsail 4 GB	2 vCPU, 4 GB RAM, 80GB SSD, 4 TB transferencia	24 USD/mes

Nota. (Amazon AWS, n.d.)

3.3 Máquina virtual

Una máquina virtual consiste en ejecutar un servidor dentro de otro equipo físico mediante un software de virtualización como VirtualBox. Esta opción permite simular

un servidor completo, con su propio sistema operativo, disco, memoria, red y servicios, sin necesidad de adquirir hardware adicional. (VMWare, n.d.)

Para el proyecto se utilizará este enfoque, ya que permite instalar y configurar el sistema operativo, XAMPP, SSH, firewall, Git y demás herramientas necesarias para un entorno controlador además de facilitar la documentación del proceso, la realización de pruebas y la corrección de errores sin afectar al equipo anfitrión.

Desde el punto de vista económico esta es la alternativa más conveniente para la etapa de desarrollo y pruebas, ya que no requiere contratar un servidor externo ni comprar hardware específico.

La principal limitación de esta opción está en el rendimiento y disponibilidad, ya que esta depende del equipo anfitrión. Si el equipo se apaga, falla o pierde la conexión, la máquina virtual deja de estar disponible. Por este motivo, aunque es una excelente opción para pruebas, no siempre es la alternativa más adecuada para producción. (VMWare, n.d.)

Concepto	Costo aproximado
Fedora Server 43	Sin costo de licencia
VirtualBox	Sin costo base / software libre
Hardware adicional	No requerido si se usa un equipo existente
Consumo eléctrico	Asociado al equipo anfitrión
Mantenimiento	Interno
Costo mensual fijo	0 USD sin contar electricidad / conexión

3.4 Bare metal

Un servidor bare metal es un servidor físico dedicado instalado directamente sobre hardware real el cual no depende de una capa de virtualización. A diferencia de una máquina virtual el sistema operativo utiliza directamente todos los recursos del equipo físico. (IBM, n.d.)

Esta alternativa ofrece mejor rendimiento, mayor control sobre el hardware y menor sobrecarga, ya que no existe intermediación de un hipervisor. Puede ser útil cuando se requiere alto rendimiento, gran volumen de datos, disponibilidad constante o configuraciones muy específicas. (IBM, n.d.)

Para un proyecto de las dimensiones planteadas este enfoque puede resultar excesivo para la etapa inicial. La aplicación funcionará como sistema web administrativo por lo que un servidor físico de alto rendimiento no sería necesario ya que la adopción de esta opción implicaría un mayor costo, una gran responsabilidad de mantenimiento y menor flexibilidad que una máquina virtual.

Concepto	Costo aproximado
Servidor físico dedicado	Alto
Sistema Operativo Fedora Server	Sin costo de licencia
Mantenimiento físico	Medio / Alto
Consumo eléctrico	Permanente
Repuestos	Variable
Escalabilidad	Menor que un virtualización

3.5 Opción seleccionada para el proyecto

Para el proyecto se selecciona la implementación mediante **máquina virtual.** Esta opción permite disponer de un servidor funcional sin adquirir hardware adicional ni contratar servicios externos, reduciendo así los costos iniciales y facilitando la realización de pruebas.

La máquina virtual permite instalar y configurar todos los componentes necesarios para el entorno de trabajo: sistema operativo, XAMPP, acceso remoto mediante SSH, firewall, Git y herramientas de administración. También permite documentar el proceso paso a paso, generar evidencias de instalación y realizar cambios sin comprometer un servidor físico.

Esta alternativa se considera adecuada para el desarrollo y validación inicial del sistema, ya que ofrece un equilibrio entre costo, control, facilidad de uso y posibilidad de replicación. Para una futura implementación definitiva se deberá evaluar si conviene mantener una arquitectura virtualizada dentro de la infraestructura institucional o una migración al servidor propio del hospital, dependiendo de los criterios de disponibilidad, seguridad y soporte requeridos.

Alternativa	Costo inicial	Costo mensual	Control	Mantenimiento
Servidor propio	Medio / Alto	Bajo / Medio	Alto	Interno
Servidor contratado	Bajo	Medio	Medio	Compartido con proveedor
Máquina virtual	Bajo	Muy bajo	Alto	Interno

Bare metal	Alto	Bajo / Medio	Muy alto	Interno

Después de comparar las alternativas se determina que la máquina virtual es la opción más conveniente para la primera etapa del proyecto. Permite trabajar con Fedora Server 43, instalar los servicios necesarios y documentar el proceso sin generar costos elevados.

El servidor propio y el bare metal quedan como alternativas posibles para una implementación definitiva, especialmente si se requiere alojar el sistema dentro de la infraestructura institucional. El servidor contratado podría ser útil por su rapidez y escalabilidad aunque debe evaluarse cuidadosamente por el tratamiento de datos, los costos recurrentes y la dependencia de un proveedor externo.

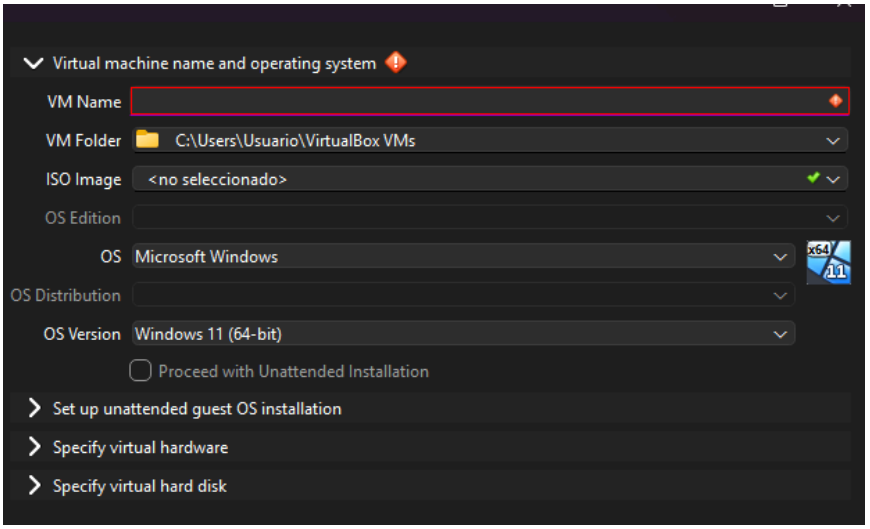
4. Creación de la máquina virtual

A continuación se detallan los pasos a seguir para la correcta creación de la máquina virtual necesaria para el entorno de desarrollo. Se tiene en cuenta que la imagen del sistema operativo ya se encuentra descargada en el dispositivo.

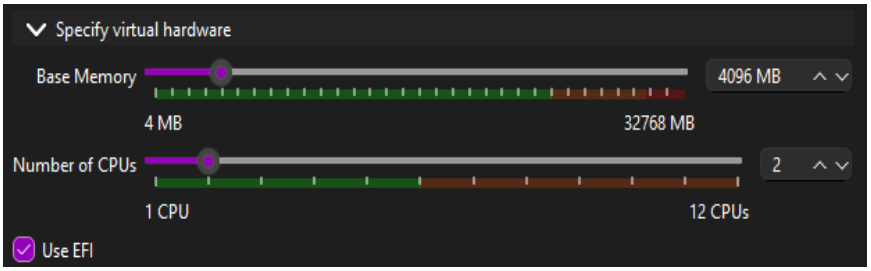
1. Dirigirse al siguiente link (https://www.virtualbox.org/) para descargar e instalar el gestor de máquinas virtuales.

2. Una vez instalado, se selecciona la opción **Nueva máquina virtual**.

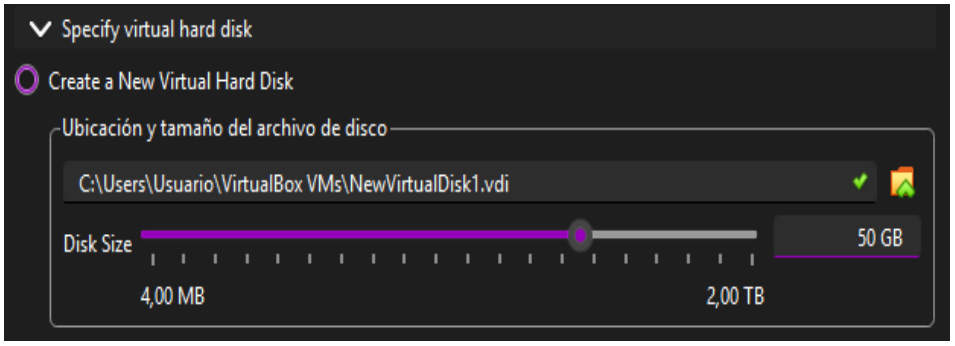
3. Se abrirá la siguiente ventana en la cual se deben rellenar los campos de nombre e imagen ISO con la imagen del sistema operativo que se va a utilizar.



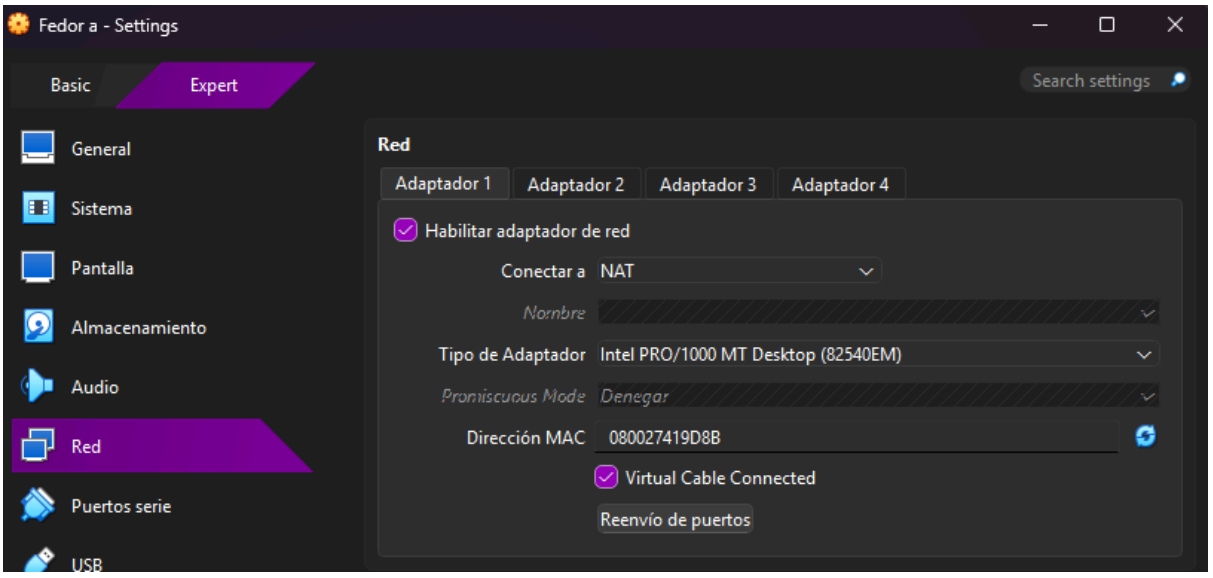
4. Dirigirse a la sección “Specify virtual hardware” donde se seleccionará el número de núcleos y la cantidad de memoria RAM de la máquina, quedando de esta manera:



5. En la sección “specify virtual hard disk” se deberá asignar el espacio que tendrá disponible nuestra máquina virtual.



6. Una vez asignado el disco virtual se debe ingresar a la configuración de la máquina virtual y seleccionar el apartado de Red. En el adaptador 1 se habilita la tarjeta de red y se selecciona la opción NAT.



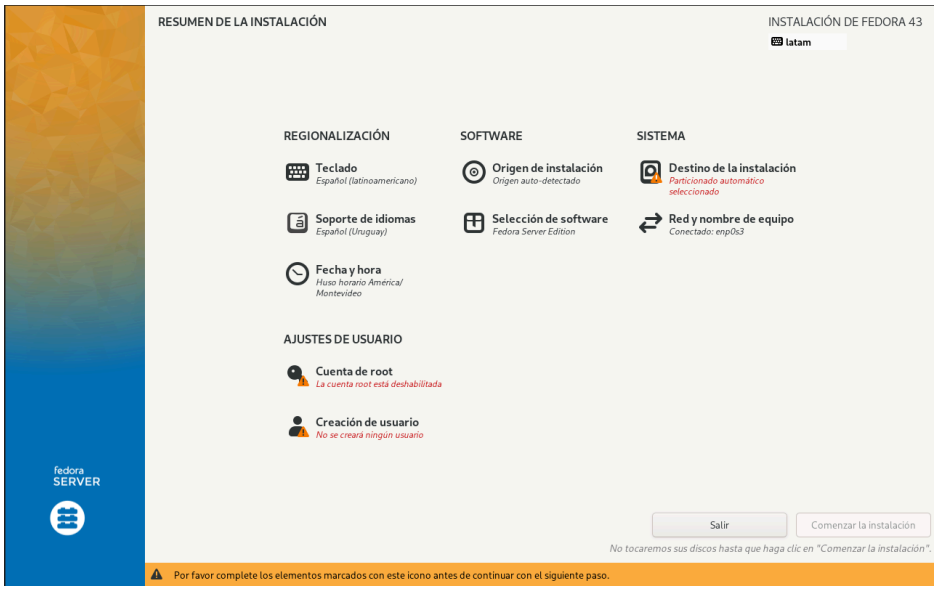
Esta configuración permite que la máquina virtual tenga acceso a internet a través del equipo anfitrión, lo que será necesario para instalar paquetes, descargar herramientas y realizar actualizaciones. Además, al utilizar esta opción la máquina virtual no queda expuesta directamente como equipo dentro de la red local.

Para esta primera etapa se utilizará el adaptador NAT, pero se configurará un adaptador puente para facilitar el acceso mediante SSH en las primeras instancias. Más avanzado el proyecto se configurarán el reenvío de puertos para habilitar el acceso a la máquina virtual a servicios como SSH o HTTP facilitando el acceso pero evitando así la exposición innecesaria de la VM.

5. Instalación de Fedora Server

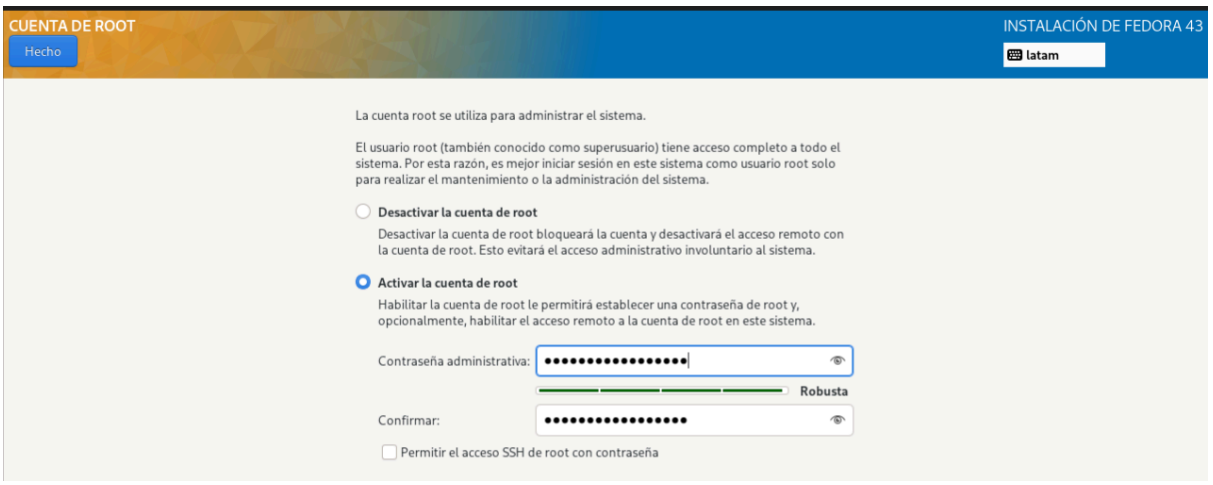
Una vez configurada la máquina virtual el siguiente paso es instalar el sistema operativo correspondiente, en este caso Fedora Server 43.

Para iniciar el proceso, se debe ejecutar la máquina virtual y seleccionar la opción **install fedora 43**. Luego de unos segundos se cargará la interfaz gráfica del instalador, desde la cual será necesario seleccionar el idioma del sistema operativo. Esta configuración permitirá continuar con las opciones principales de instalación. (*Fedora Server Interactive Local Installation :: Fedora Docs*, 2026)



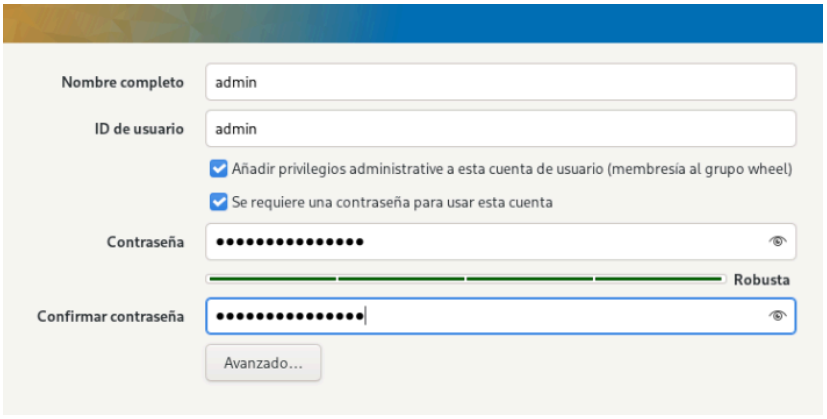
A continuación se procederá a configurar las cuentas de usuario del sistema. En primer lugar se creará la cuenta root, que corresponde al usuario con privilegios máximos dentro del sistema operativo. Para ello se debe acceder a la sección **ajustes de usuario > cuenta de root.** En esta pantalla se deberá marcar la opción **activar cuenta de root** e ingresar una contraseña segura en los campos correspondientes. (*Fedora Server Interactive Local Installation :: Fedora Docs*, 2026)

Por motivos de seguridad no se recomienda habilitar el acceso remoto por SSH para el usuario root, ya que esto podría aumentar el riesgo de accesos no autorizados al servidor. La administración remota deberá realizarse preferentemente mediante un usuario administrador con permisos elevados. (*Fedora Server Interactive Local Installation :: Fedora Docs*, 2026)

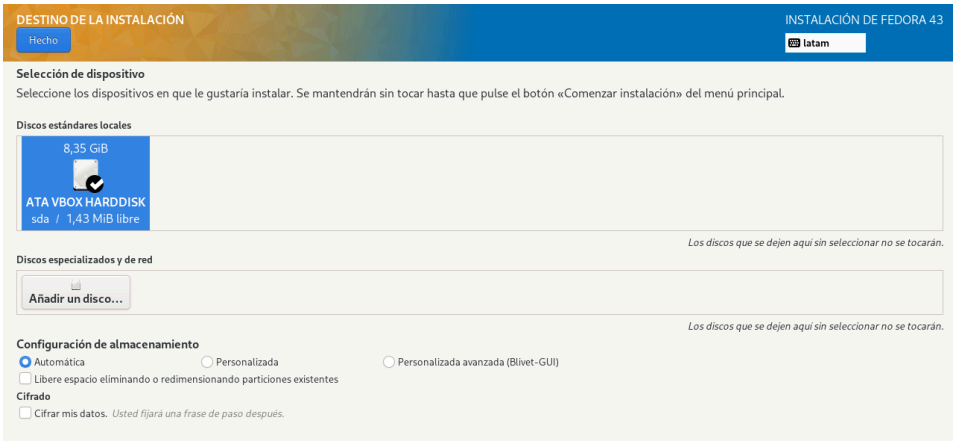


Una vez configurada la cuenta root se debe regresar al menú principal del instalador para ingresar en la opción **ajustes de usuario > creación de usuario** con el fin de crear a nuestro usuario administrador. (Boy et al., 2026)

En esta sección se deben completar los campos correspondientes al nombre de usuario y contraseña. Además, se debe seleccionar la opción que permite agregar este usuario al grupo de administradores, identificado en fedora como el grupo wheel. De esta forma el usuario podrá ejecutar tareas administrativas mediante el comando **sudo** sin necesidad de utilizar directamente la cuenta root. (*Fedora Server Interactive Local Installation :: Fedora Docs*, 2026)



Posteriormente, se debe configurar el destino de instalación del sistema operativo. Para ello se debe acceder a la sección **Destino de instalación** y seleccionar el disco virtual asignado previamente a la máquina. Es importante verificar que el disco seleccionado cuente con espacio suficiente para la instalación del sistema y los otros servicios que se utilizarán a lo largo del proyecto. (*Fedora Server Interactive Local Installation :: Fedora Docs*, 2026)



Una vez configurada la cuenta root, creada la cuenta de administrador y seleccionado el disco de instalación, el instalador habilitará en el menú principal la opción para comenzar el proceso. Al presionar dicho botón, Fedora Server 43 iniciará la instalación del sistema operativo aplicando las configuraciones definidas anteriormente.

Durante este proceso el instalador copiará los archivos necesarios, configurará el sistema base y creará las cuentas de usuario establecidas. Al finalizar se deberá reiniciar la máquina virtual para iniciar por primera vez Fedora Server desde el disco instalado.

Luego del reinicio el sistema estará listo para iniciar sesión con el usuario administrador creado durante la instalación y continuar con las configuraciones necesarias del servidor.

6. Actualización inicial del sistema y política de actualizaciones

Después de iniciar sesión en Fedora Server 43 se recomienda actualizar el sistema operativo antes de instalar cualquier servicio adicional. Lo que permite incorporar correcciones, mejoras de estabilidad y actualizaciones de seguridad.

Para actualizar el sistema se debe ejecutar este comando: **dnf update -y** (la flag -y acepta todas las confirmaciones de manera automática).
Una vez finalizado el proceso se recomienda reiniciar el servidor para asegurar que todos los cambios se apliquen correctamente. (Fedora Project, 2026)

Mantener actualizado el sistema operativo es una tarea fundamental. Estas actualizaciones permiten corregir errores, cerrar vulnerabilidades de seguridad y mejorar el funcionamiento general del entorno. Sin embargo, gestionar estas actualizaciones de forma manual puede ser tedioso por lo que Fedora Server 43 permite realizar esta tarea mediante la herramienta dnf5-plugin-automatic, la cual ofrece distintos modos de automatización a la hora de mantener el dispositivo al día.

Por ende se plantea el uso de esta herramienta para el manejo de las futuras actualizaciones pertinentes. Se detallan, para ello, los pasos a seguir para configurar dnf5-plugin-automatic de manera que mande un correo al administrador del sistema cuando haya actualizaciones disponibles, dejando así en manos de éste la toma de decisión y evitando cambios automáticos que puedan afectar el funcionamiento del servidor.

Para instalar la herramienta se ejecuta: **dnf install dnf5-plugin-automatic -y**

Luego se verifica que exista el archivo de configuración: **/usr/share/dnf5/dnf5-plugins/automatic.conf**

Una vez verificado el archivo se copia éste al directorio de configuración del sistema: **cp /usr/share/dnf5/dnf5-plugins/automatic.conf /etc/dnf/automatic.conf**

A continuación se edita el archivo con vim: **vim /etc/dnf/automatic.conf**

Dentro del archivo se busca una sección [commands], debajo de ella se encuentran dos campos que debemos modificar **download_updates** y **apply_updates** ambos deben estar en **no.**
Con esta configuración el sistema no descargará ni instalará actualizaciones automáticamente. Solo realizará la comprobación y emitirá el aviso correspondiente.

Luego se debe configurar el método de notificación. En el mismo archivo se busca la sección [emitters] y se indica el uso de correo electrónico: **emit_via = email**.

Después, en la sección [email] se configuran los datos básicos del aviso:
email_from = root@host.example.com
email_to = correo_admin@ejemplo.com

Los campos mencionados permiten definir el remitente y destinatario de los avisos generados por dnf5-plugin-automatic. Para que estos correos se envíen correctamente, el servidor debe contar con un servicio de correo configurado, ya sea mediante SMTP local o un servidor externo. En esta etapa se dejan definidos los parámetros básicos de notificación quedando el envío efectivo condicionado a la configuración posterior del servicio SMTP.

Una vez configurado el archivo se habilita el temporizador de actualizaciones automáticas: **systemctl enable –now dnf5-automatic.timer**

Para verificar que quedó activo se ejecuta: **systemctl status dnf5-automatic.timer**

La configuración aplicada permite que el sistema operativo revise las actualizaciones disponibles sin instalarlas automáticamente. Esta decisión evita cambios inesperados en el servidor y permite que el administrador evalúe previamente qué actualizaciones aplicar y en qué momento hacerlo.

7. Configuración de red inicial

Una vez instalado el sistema operativo se debe verificar que el servidor cuente con conexión de red, dado que será necesaria para instalar paquetes, acceder al servidor de forma remota y probar el funcionamiento de la aplicación web desde otros dispositivos.

Para obtener la dirección IP del servidor se utiliza: **ip a**
La dirección IPv4 obtenida será utilizada posteriormente para acceder al servidor desde un navegador web o mediante una conexión SSH.

Es importante comprobar que el servidor tenga salida a internet y que pueda resolver nombres de dominio, para ello se usa el comando **ping** ya sea con una dirección ip como 8.8.8.8 o directamente el nombre de dominio, por ejemplo google.com.

Para el entorno de pruebas se recomienda utilizar la máquina virtual con un adaptador puente dado que facilita el acceso al servidor desde otros dispositivos conectados a la misma red, siendo extremadamente útil a la hora de probar la aplicación desde distintos dispositivos.

8. Configuración básica de SSH

Este servicio será utilizado para administrar el servidor de forma remota. Esto permite acceder a la terminal del servidor desde otro dispositivo sin la necesidad de utilizar directamente la consola de la máquina virtual.

En Fedora Server 43 el servicio SSH viene instalado como parte del entorno base del servidor. Por este motivo, en esta etapa no es necesario instalar nuevamente el paquete, sino verificar que el servicio se encuentre disponible y activarlo en caso de que no sea el caso. (Fedora Project, 2026)

Para comprobar el estado del servicio se ejecuta: **systemctl status sshd**

Si el servicio no se encuentra activo se puede iniciar con el siguiente comando: **systemctl enable –now sshd**

Una vez activo el acceso remoto se realiza desde otro equipo mediante el siguiente formato: **ssh usuario@ip_del_servidor,** por ejemplo ssh admin@192.168.1.4. (Fedora Project, 2026)

Esta configuración permite administrar el servidor de manera cómoda y es una base necesaria para tareas posteriores de mantenimiento, revisión de servicios, actualización del sistema y administración de la aplicación web.

Por seguridad no se recomienda permitir el acceso remoto directo con el usuario root. La administración debe realizarse con un usuario normal con permisos de administrador mediante **sudo**.

9. Configuración básica del firewall

Fedora Server 43 utiliza **firewalld** como herramienta para administrar el firewall del sistema. La función de este es controlar qué servicios pueden recibir conexiones desde la red, permitiendo así reducir la exposición del servidor y habilitar únicamente los accesos necesarios. (Fedora Project, 2026)

En el contexto del proyecto, se deben permitir los servicios básicos relacionados con la administración remota y el funcionamiento del servidor web mediante XAMPP.

En primera instancia se debe verificar que firewalld esté instalado y activo con el comando **systemctl status firewalld**, si el servicio no se encuentra activo se habilita mediante **systemctl enable –now firewalld**. (Fedora Project, 2026)

Con el firewall ya activado se procede a habilitar el acceso por ssh mediante **firewall-cmd –permanent –add-service=ssh**. Para el resto de servicios como http el comando es idéntico salvo por el valor asignado a la flag **–add-service**. (Fedora Project, 2026)

Una vez agregadas las reglas es necesario recargar la configuración, para ello se usa: **firewall-cmd –reload**.

Finalmente verificamos la configuración aplicada con el comando **firewall-cmd –list-all**. Lo que nos debe devolver los servicios activos del firewall, en este caso SSH, HTTP y Cockpit.

Con esta configuración el servidor permite la administración remota mediante SSH y el acceso web a XAMPP desde otros dispositivos de la red. Al mismo tiempo se mantiene cerrado cualquier otro puerto que no sea necesario para esta etapa del proyecto.

10. Instalación de XAMPP

Para ejecutar la aplicación web se utilizará XAMPP para linux ya que integra en una sola instalación los servicios necesarios para el entorno inicial: Apache, MariaDB y PHP. Esto permite simplificar la configuración del servidor durante la etapa de pruebas y facilita la verificación del correcto funcionamiento de la aplicación.

El instalador de XAMPP debe descargarse desde el sitio oficial de Apache Friends. El archivo correspondiente para Linux suele tener extensión .run
Por ejemplo, en este momento el comando para descargar la versión 8.2.12 es el siguiente:
wget
https://sourceforge.net/projects/xampp/files/XAMPP%20Linux/8.2.12/xampp-linux-x64-8.2.12-0-installer.run (Mutai, 2026)

Se recomienda crear una carpeta de descargas en la cual posicionarse antes de ejecutar el comando anterior.

Una vez completada la descarga se creará en nuestra carpeta un instalador con un nombre similar a este: **xampp-linux-x64-8.2.12-0-installer.run**

Antes de ejecutarlo se deben asignar permisos con el comando **chmod +x** y el nombre del archivo. Una vez asignados se procede a iniciar el instalador con permisos de administrador: **sudo ./xampp-linux-x64-8.2.12-0-installer.run** (Mutai, 2026)

Durante el proceso de instalación se deben aceptar las opciones indicadas por el asistente y al finalizar XAMPP quedará instalado por defecto en **/opt/lampp**
Esta ubicación será utilizada posteriormente para iniciar los servicios pertinentes, ubicar archivos del proyecto y verificar el funcionamiento de Apache, PHP y MariaDB. (Mutai, 2026)

Es importante aclarar que XAMPP se utilizó durante las primeras etapas del proyecto como entorno de desarrollo y pruebas, debido a su facilidad para integrar Apache, PHP y MariaDB en una única instalación. Sin embargo, con el avance del proyecto se optó por implementar una arquitectura basada en contenedores mediante Docker Compose, permitiendo administrar estos servicios de forma independiente y facilitando el despliegue de la aplicación. Para una futura implementación en los servidores del Hospital de Clínicas, será necesario evaluar esta solución junto al Departamento Técnico de Informática, considerando los requisitos de seguridad, mantenimiento y las políticas de infraestructura de la institución.

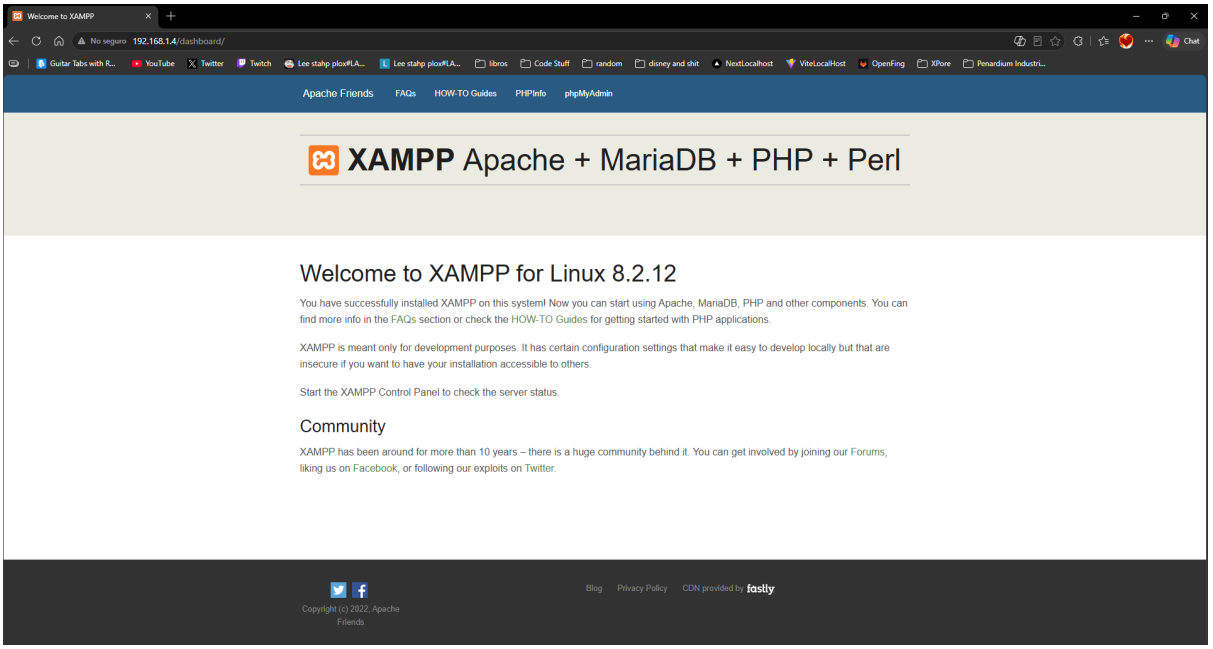
11. Inicio y verificación de XAMPP

Una vez finalizada la instalación se deben iniciar los servicios incluidos en XAMPP. Para ello se utiliza el script de administración ubicado en **/opt/lampp.** Podemos utilizar **sudo /opt/lampp/lampp start** para iniciar los servicios principales como MariaDB y Apache. (Mutai, 2026)

Durante el primer intento de inicio puede aparecer un mensaje indicando que se requiere una librería de compatibilidad de 32 bits. En Fedora Server 43 este mensaje puede deberse a la falta de compatibilidad necesarias para ejecutar algunos binarios incluidos en XAMPP. Para corregirlo se instalan los paquetes **libnsl** y **libxcrypt-compat**. (Mutai, 2026)

Una vez instalados estos paquetes de ser necesarios, se debe volver a ejecutar el comando de inicio de XAMPP para después comprobar su estado con el siguiente comando: **sudo /opt/lampp/lampp status**. El cual muestra el estado actual de los servicios. (Mutai, 2026)
En caso de necesitar detener XAMPP se utiliza: **sudo /opt/lampp/lampp stop**, también se puede usar **restart** como último argumento si se quiere reiniciar los servicios que se encuentren activos. (Mutai, 2026)

Luego de iniciar XAMPP se debe probar el acceso desde un navegador web, en este caso desde el equipo anfitrión. Para ello se debe obtener la dirección IP del servidor: **ip a**, y una vez identificada, acceder a ella a través del navegador antes mencionado. Si se siguieron los pasos correctamente esto nos debería mostrar la página principal de Apache.



12. Verificación de Apache, PHP y MariaDB

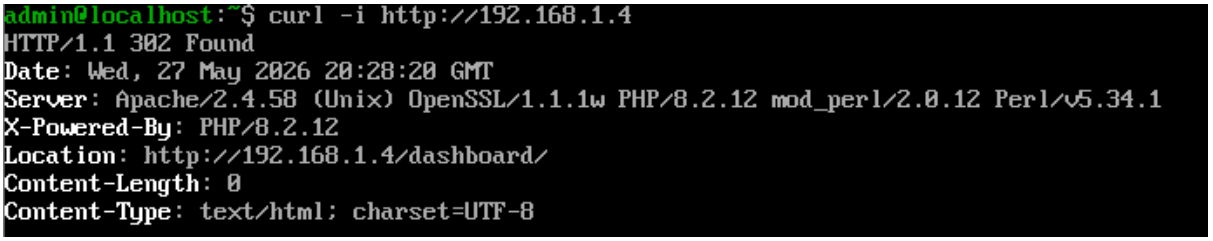
Una vez iniciado XAMPP se debe comprobar que sus componentes principales funcionen correctamente. Se detalla a continuación la manera de verificar el funcionamiento de Apache, MariaDB y PHP.

12.1 Verificación de Apache

Apache se puede verificar accediendo desde un navegador en el anfitrión a la dirección IP del servidor. Un ejemplo de esto sería http://192.168.1.4, si al ingresar a esta dirección se muestra la página inicial de XAMPP se confirma que el servidor web Apache está activo y funciona correctamente.

También se puede realizar una prueba desde la propia terminal a través del comando **curl -i http://localhost**, este comando devuelve únicamente las cabeceras

de la respuesta HTTP. Si Apache funciona correctamente se debería ver algo parecido a la siguiente imagen.



12.2 Verificación de PHP

Para comprobar que PHP se ejecuta correctamente se creará un archivo de prueba dentro del directorio público de XAMPP, siendo la dirección de este **/opt/lampp/htdocs.** Para ello se crea un archivo info.php utilizando vim mediante el siguiente comando: **vim /opt/lampp/htdocs/info.php**, puede ser necesario utilizar privilegios de superusuario para poder crear el documento.

Una vez en el archivo ingresamos el siguiente código:

```
<?php
phpinfo()
?>
```

Después se guarda el archivo y se procede a acceder desde el navegador de nuestro equipo anfitrión mediante el siguiente formato de URL: **http://IP_DEL_SERVIDOR/info.php**, por ejemplo http://192.168.1.4/info.php.

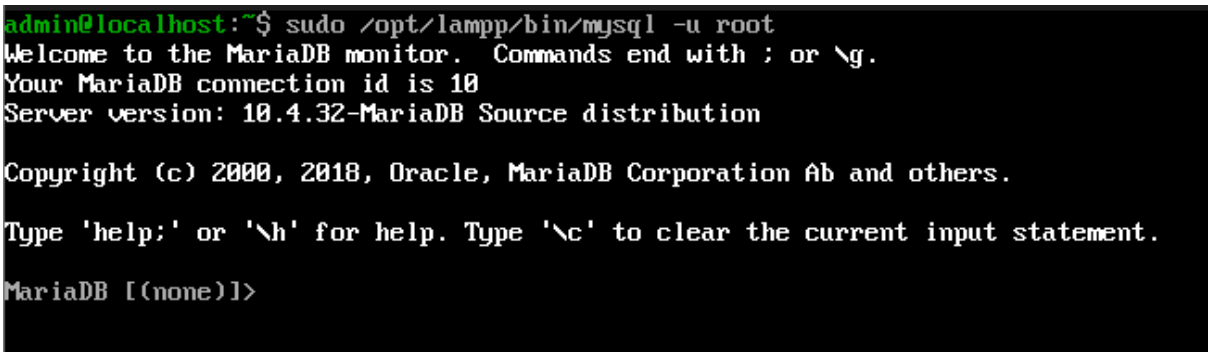
Si PHP funciona correctamente se mostrará una página con información técnica sobre la versión instalada y sus módulos activos.

Una vez finalizada la comprobación es importante eliminar este archivo dado que expone información interna del servidor.

12.3 Verificación de MariaDB

Para la verificación de MariaDB es necesario acceder a la consola del motor de base de datos, en esta prueba se accederá con el usuario root (el cual no es el mismo usuario root que gestiona el sistema operativo). Para ello se ejecuta **sudo /opt/lampp/bin/mysql -u root.**

Si el acceso es correcto se abrirá la siguiente consola:



Desde la cual se podrán ejecutar consultas SQL como **SHOW DATABASES;** la cual mostrará las bases de datos disponibles dentro del sistema demostrando así el correcto funcionamiento de MariaDB.

Con estas pruebas queda demostrado que XAMPP y sus servicios principales se encuentran operativos y disponibles.

13. Instalación y configuración de Git

Durante el proyecto se utilizará Git como herramienta de control de versiones del proyecto dado que permite mantener un historial de cambios, organizar el trabajo del equipo y facilitar el despliegue del código dentro del servidor.

Para instalar Git se ejecuta **sudo dnf install git**, con la flag opcional -y si se quiere aceptar automáticamente la instalación.

Una vez finalizada podemos comprobar la correcta instalación con el comando **git –version**, el cual nos devuelve el número de versión de Git que se ha instalado. (Git, n.d.)

Después de verificar la instalación es importante configurar los datos básicos del usuario que realizará los cambios en el repositorio. (Git, n.d.) Para ello existen dos comandos:
git config –global user.name “nombre de usuario”
git config –global user.email “correo@admin.com”

Una vez configurados los datos anteriormente mencionados se procede a comprobar que éstos fueron aplicados correctamente mediante el comando: **git config –global –list.** El cual debería listar tanto el nombre de usuario como el correo asignado previamente. (Git, n.d.)

14. Pruebas y evidencias de la instalación

Si bien se ha ido dejando evidencia de la correcta instalación de los paquetes y herramientas necesarios para el funcionamiento del servidor durante el presente manual, esta sección tiene como objetivo unificar en un solo lugar dichas evidencias para facilitar la revisión final del entorno.

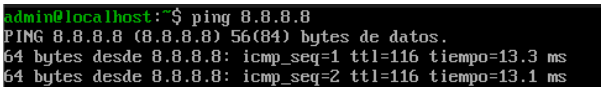
La idea de esta sección es comprobar de manera ordenada, que todo lo instalado hasta este punto funciona correctamente. Para ello se realizarán pruebas básicas sobre el sistema operativo, la conexión de red, SSH, firewall, XAMPP, Apache; PHP, MariaDB y Git.

Cada prueba deberá acompañarse de su respectiva captura de pantalla del resultado obtenido. Esto permite demostrar no solo que el servidor fue instalado sino que también se encuentra funcional y disponible para continuar con el despliegue del sistema.

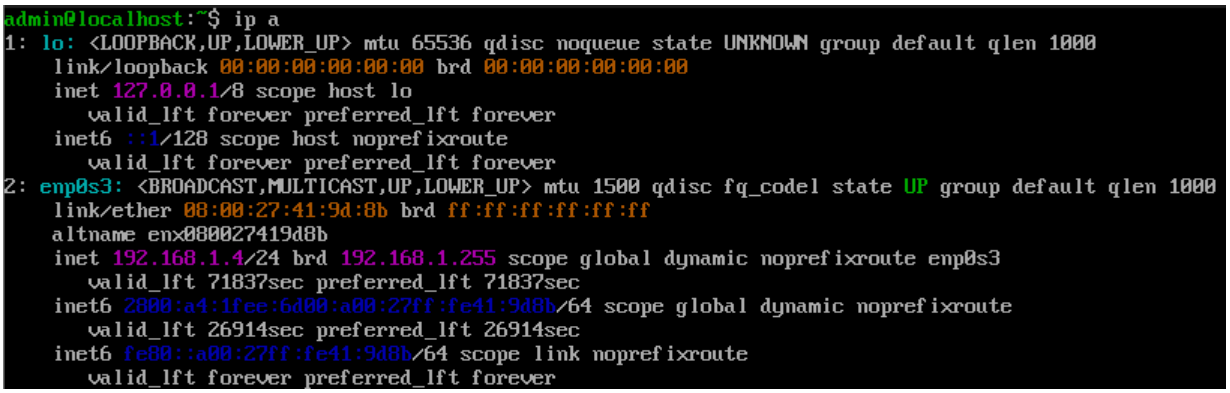
- **Versión del sistema operativo:** se verifica con el comando **cat /etc/fedora-release**. El resultado esperado es que muestre Fedora 43



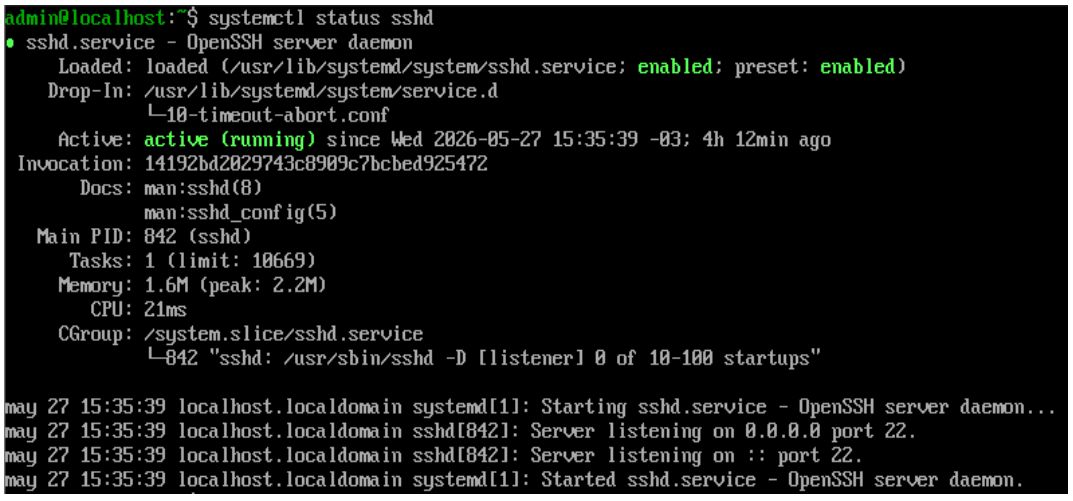
- **Conectividad de red:** se comprueba con el comando **ping 8.8.8.8**. El resultado esperado es recibir respuestas correctamente, lo que confirma que el servidor tiene conexión a internet.



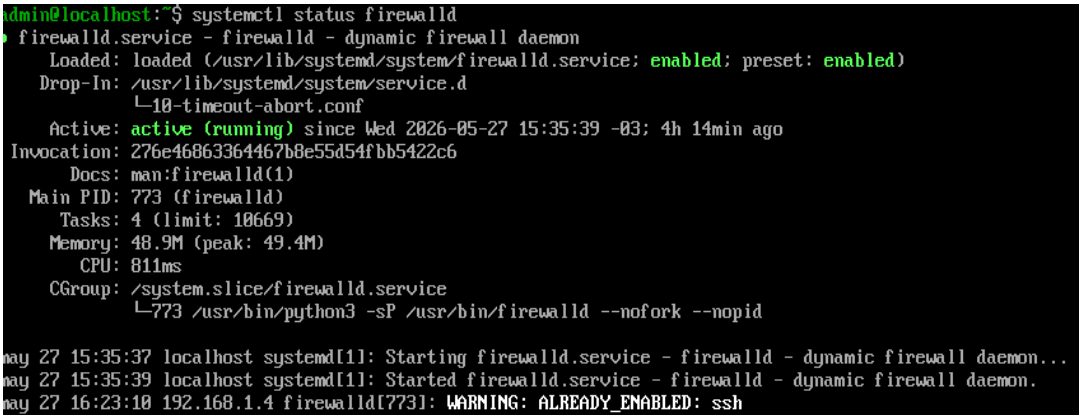
- **Dirección IP del servidor:** se obtiene mediante el comando **ip a**. El resultado esperado es identificar una dirección IPv4 válida asignada al servidor.



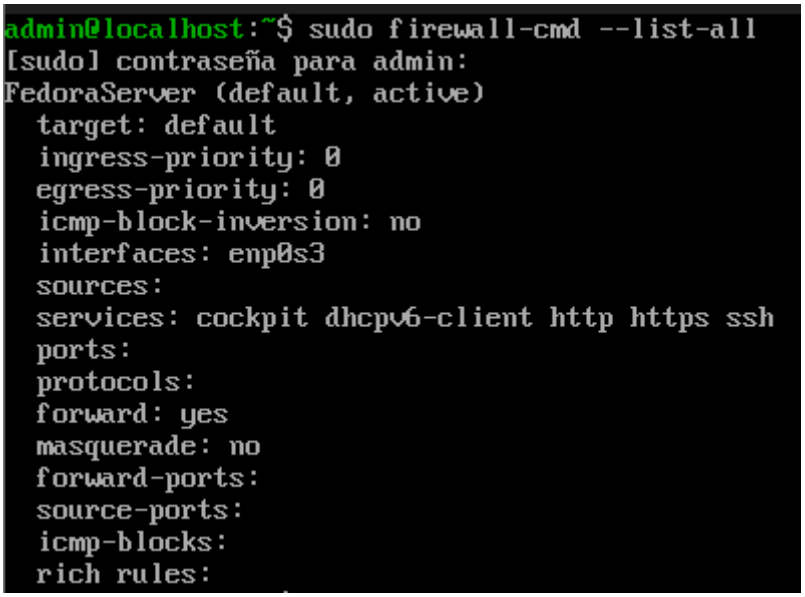
- **Estado de SSH:** se verifica con **systemctl status sshd**. El resultado esperado es que el servicio figure como activo.



- **Estado del firewall:** se comprueba con **systemctl status firewalld**. El resultado esperado es que el servicio se encuentre activo.



- **Reglas del firewall:** se revisan con **sudo firewall-cmd –list-all**. El resultado esperado es que se encuentren habilitados como mínimo los servicios HTTP y SSH.



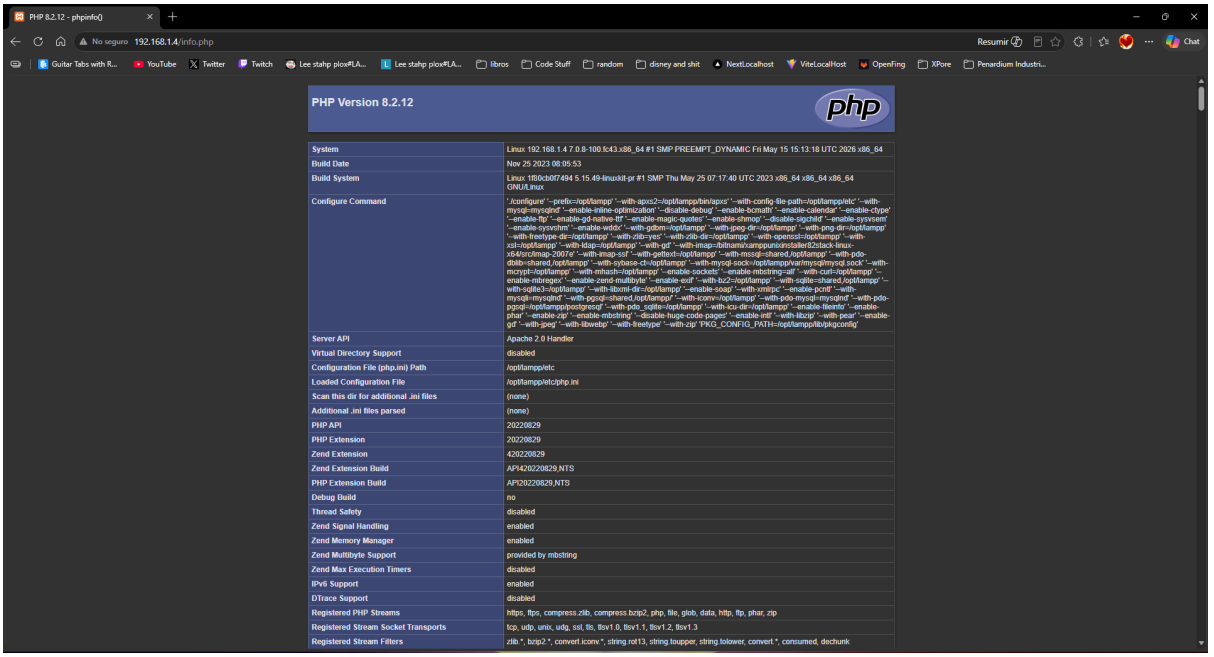
- **Estado de XAMPP:** se verifica con sudo **/opt/lampp/lampp status**. El resultado esperado es que Apache y MariaDB se encuentren activos.

```
admin@localhost:~$ sudo /opt/lampp/lampp status
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
egrep: warning: egrep is obsolescent; using grep -E
Version: XAMPP for Linux 8.2.12-0
egrep: warning: egrep is obsolescent; using grep -E
Apache is running.
egrep: warning: egrep is obsolescent; using grep -E
MySQL is running.
egrep: warning: egrep is obsolescent; using grep -E
ProFTPD is running.
```

- **Respuesta de Apache:** se comprueba con **curl -I** http://localhost. El resultado esperado es obtener una respuesta HTTP válida.

```
admin@localhost:~$ curl -I http://localhost
HTTP/1.1 302 Found
Date: Wed, 27 May 2026 22:56:22 GMT
Server: Apache/2.4.58 (Unix) OpenSSL/1.1.1w PHP/8.2.12 mod_perl/2.0.12 Perl/v5.34.1
X-Powered-By: PHP/8.2.12
Location: http://localhost/dashboard/
Content-Type: text/html; charset=UTF-8
```

- **Redirección de PHP:** se realiza accediendo desde el navegador a http://IP_DEL_SERVIDOR/info.php. El resultado esperado es que se muestre la página de información de PHP.



- **Verificación de MariaDB:** se comprueba con **sudo /opt/lampp/bin/mysql -u root**. El resultado esperado es que se abra la consola de MariaDB.

```
admin@localhost:~$ sudo /opt/lampp/bin/mysql -u root
[sudo] contraseña para admin:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 11
Server version: 10.4.32-MariaDB Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

- **Verificación de Git:** se verifica mediante el comando **git –version**. El resultado esperado es que se muestre la versión instalada de Git.

```
admin@localhost:~$ git --version
git version 2.54.0
```

14. Consideraciones de seguridad iniciales

Durante la instalación del servidor se aplican medidas básicas de seguridad orientadas a reducir riesgos desde la primera configuración. Si bien esta etapa corresponde a un entorno de pruebas es importante establecer buenas prácticas desde el inicio con el fin de facilitar una futura implementación más segura.

En primer lugar, no se recomienda utilizar el usuario root para la administración remota del servidor. Aunque la cuenta puede existir en el sistema, las tareas administrativas deben realizarse a través de un usuario normal con permisos de superusuario con el fin de reducir el riesgo asociado al uso directo de la cuenta con mayores privilegios.

El acceso por SSH debe mantenerse habilitado únicamente a usuarios autorizados. En una configuración posterior se recomienda reforzar este servicio mediante medidas como el uso de claves o la revisión de los intentos de inicio.

Se configura firewalld para permitir solo los servicios necesarios en esta etapa. En este caso SSH para administración remota y HTTP para el acceso al servidor web. Cualquier otro puerto que no sea requerido debe permanecer cerrado.

Mantener el sistema actualizado también es una medida importante. Para ello se configura dnf5-plugin-automatic en modo de notificación, permitiendo que el administrador del sistema reciba avisos sobre las actualizaciones disponibles sin que estas se instalen automáticamente. Esta decisión evita cambios inesperados en el servidor y permite aplicar actualizaciones de forma controlada.

En relación con XAMPP debe tenerse en cuenta para su implementación definitiva la revisión de sus configuraciones de seguridad, sobre todo las relacionadas con MariaDB. Será necesario establecer contraseñas seguras, evitar el uso de root para la conexión y crear usuarios con permisos limitados según las funciones requeridas para interactuar con la base de datos. También deberán deshabilitarse los servicios que no sean necesarios.

Se plantean entonces las siguientes medidas:

- Administrar el servidor con un usuario normal con permisos de superusuario
- No habilitar el acceso remoto directo para root
- Mantener activo el firewall
- Permitir únicamente los servicios necesarios
- Mantener el sistema actualizado
- Evitar el uso de contraseñas débiles

Estas acciones establecen una base inicial adecuada para el servidor y preparan éste para configuraciones más estrictas en etapas posteriores.

15. Vulnerabilidades e incidentes de seguridad pasados

En el siguiente apartado se pretenden identificar incidentes de seguridad y vulnerabilidades pasadas asociados a los componentes instalados previamente en el servidor. Estas vulnerabilidades corresponden a fallos publicados en bases de datos reconocidas o en los sitios oficiales de los propios proyectos. Su inclusión no significa que el servidor instalado sea actualmente vulnerable sino que permite reconocer riesgos posibles y definir medidas de mitigación adecuadas a la situación.

Entre las fuentes consultadas para la sección se encuentran las páginas oficiales de CVE y NIST, además de la propia documentación de los proyectos implementados.

15.1 Apache HTTP Server

CVE-2023-43622: Esta vulnerabilidad de denegación de servicio se producía cuando un atacante abría una conexión HTTP/2 con un tamaño de ventana inicial de cero, lo que permitía bloquear el manejo de esa conexión de forma indefinida. Esto podría agotar los recursos del servidor en un ataque con un comportamiento similar al del tipo “slow loris”. La vulnerabilidad afectaba a Apache HTTP Server desde la versión 2.4.55 hasta la 2.4.57. Por lo que para evitar este tipo de ataque se recomienda actualizar a la versión 2.4.58 (instalada en el servidor), la cual soluciona esta vulnerabilidad. (National Vulnerability Database (NVD) NIST, 2023)

CVE-2023-38709: Esta vulnerabilidad está relacionada con la validación incorrecta de inputs, lo que permite, lo que permite a actores maliciosos dividir respuestas HTTP, generando un problema conocido como *HTTP response splitting*. Este tipo de vulnerabilidad puede derivar en respuestas inesperadas del servidor y facilitar otros ataques como la manipulación de cabeceras o el envenenamiento de caché. Esta vulnerabilidad afectaba los servicios de Apache HTTP Server hasta la versión 2.4.58. (National Vulnerability Database (NVD) NIST, 2024)

CVE-2022-37436: Es una vulnerabilidad que afectaba desde la versión 2.4 hasta la 2.4.54. El problema se producía mediante una cabecera manipulada, la cual era capaz de provocar una lectura de memoria o la escritura de un byte cero fuera del valor enviado en la cabecera. Como consecuencia de ese proceso el servidor podía fallar y cerrarse, generando una denegación del servicio. (National Vulnerability Database (NVD) NIST, 2023)

15.2 PHP

CVE-2026-7568: Esta vulnerabilidad está relacionada con la función *metaphone* de PHP. En determinados casos, al recibir una cadena superior a 2.147.483.647 bytes, se podía producir un error que derivaba en un comportamiento no definido en el programa. Esto podría ocasionar una lectura fuera de límites, el cierre del proceso o el acceso a zonas de memoria no previstas. La vulnerabilidad afecta a las versiones 8.2 de PHP anteriores a la versión 8.2.31. (National Vulnerability Database (NVD) NIST, 2026)

CVE-2026-7262: Es una vulnerabilidad que afecta principalmente a las versiones 8.2 de PHP anteriores a la versión 8.2.31. El fallo se produce cuando un servidor

SOAP tiene configurado un typemap y durante el proceso de decodificación se comprueba una variable incorrecta lo que genera una referencia a un puntero nulo y causa la caída del proceso. (*NVD - CVE-2026-7262*, 2026)

CVE-2026-7261: Esta vulnerabilidad se relaciona con el manejo persistente de sesiones en servidores SOAP. Cuando se utiliza SOAP_PERSISTANCE_SESSION ocurre un error que puede provocar que PHP libere un objeto pero conserve una referencia a este. Esta situación genera una condición de *use-after-free* que puede ocasionar corrupción de memoria, filtración de información o cierres inesperados del proceso. (*NVD - CVE-2026-7261*, 2026)

CVE-2026-7259: Esta vulnerabilidad está relacionada con el manejo de codificaciones. Una incompatibilidad entre Oniguruma y mbfl puede provocar que PHP intente acceder a un puntero nulo durante el uso de la función mb_regex_encoding. Esto puede generar que el proceso se cierre de manera inesperada. (National Vulnerability Database (NVD) NIST, 2026)

15.3 MariaDB

CVE-2023-22084: Es una vulnerabilidad corregida en la versión 10.4.32 de MariaDB. El problema se encontraba relacionado con el componente InnoDB el cual permitía a un atacante con privilegios elevados y acceso a la red provocar un bloqueo o una caída del servidor de bases de datos. En el caso del servidor instalado se utiliza la versión 10.4.32 mencionada anteriormente dado que corrige esta vulnerabilidad. (NVD, 2023)

CVE-2022-47015: Esta vulnerabilidad afectaba a MariaDB hasta la versión 10.3.34 podía causar una denegación del servicio al afectar una funcion *spider_db_mbase::print_warnings* la cual hacía referencia a un puntero nulo crasheando el servicio. (National Vulnerability Database (NIST), 2023)

CVE-2023-26785: Se descubrió una vulnerabilidad que afectaba a MariaDB en la versión 10.5 la cual permite ejecutar código de forma remota mediante el uso de funciones UDF. El problema consistía en cargar código desde un archivo compartido y luego invocarlo mediante una sentencia CREATE FUNCTION. Esta vulnerabilidad fue discutida por MariaDB ya que se necesitan privilegios previos para poder explotarla, por lo que no lo consideraban una ruptura real de los límites de seguridad. (National Vulnerability Database (NIST), 2024) Aun así, esto demuestra la importancia de restringir los privilegios de los usuarios de base de datos y evitar que cuentas no autorizadas puedan crear funciones o cargar archivos externos.

15.4 OpenSSH

CVE-2025-48416: Esta vulnerabilidad está relacionada con un servicio OpenSSH expuesto en el puerto 22. El problema principal es la entrada de una entrada fija para el usuario root en el archivo /etc/shadow. Aunque la configuración por defecto de SSH tiene deshabilitado el inicio de sesión con root, esta protección puede ser modificada o evadida de distintas maneras. (National Vulnerability Database (NIST), 2025)

CVE-2023-51767: Esta vulnerabilidad afecta a OpenSSH hasta la versión 10.0 y está relacionada con un posible bypass de autenticación mediante ataques RowHammer sobre la memoria DRAM del servidor. El problema se basa en la posibilidad de alterar un bit del valor interno utilizado durante la autenticación de contraseña. (National Vulnerability Database (NIST), 2023)

CVE-2025-26466: Esta vulnerabilidad afecta al servidor OpenSSH en determinadas versiones anteriores a su corrección en la 10.0. El fallo se produce porque durante el intercambio de claves, cada paquete ping recibido genera un paquete pong que queda almacenado en memoria hasta finalizar la negociación. Un cliente malicioso podría enviar múltiples paquetes de este tipo aumentando el consumo de memoria del servidor afectando así su disponibilidad. (National Vulnerability Database (NIST), 2025)

15.5 Git

CVE-2025-48384: Esta vulnerabilidad afecta a versiones anteriores a la versión 2.54.0 de Git y está relacionada con el manejo incorrecto de caracteres de retorno en archivos de configuración. Al clonar un repositorio malicioso con submódulos, Git podría interpretar una ruta modificada y escribir archivos en una ubicación incorrecta. En ciertos casos esto podría permitir la ejecución de código malicioso luego del clonado. (National Vulnerability Database (NIST), 2025)

CVE-2024-50349: Esta vulnerabilidad está relacionada con una falta de sanitización de secuencias de escape en URLs utilizadas por Git al solicitar credenciales. Un

atacante podría crear una URL especialmente manipulada para confundir al usuario en la terminal, haciendo que creyera estar ingresando sus credenciales para Git cuando en realidad estas estaban siendo robadas. Fue corregida previa a la versión 2.54.0. (National Vulnerability Database (NIST), 2025)

CVE-2024-52005: Esta vulnerabilidad afecta el manejo de mensajes recibidos por Git durante operaciones como clone, fetch o push. El problema se produce porque Git no neutralizaba correctamente determinadas secuencias ANSI recibidas por la comunicación. Esto podría permitir que un servidor remoto malicioso mostrara mensajes engañosos en la terminal del usuario. (National Vulnerability Database (NIIST), 2025)

16. Configuración de red y acceso remoto al servidor

16.1 Introducción

La configuración de red y del acceso remoto constituye una parte fundamental de la puesta en funcionamiento del servidor utilizado en el proyecto S.I.G.S.M. Para facilitar su administración y garantizar que pueda ser identificado de forma estable dentro de la red, se configurará una dirección IP estática, evitando que la dirección del servidor cambie automáticamente y permitiendo acceder a sus servicios siempre mediante una dirección conocida.

La configuración de la red se realizará mediante la herramienta nmtui, incluida en el sistema GNU/Linux, que permite administrar de forma interactiva las conexiones de red gestionadas por NetworkManager. A través de esta herramienta se establecerán manualmente los parámetros necesarios para la configuración de la red utilizada por

el servidor y posteriormente se verificará que estos hayan sido aplicados correctamente.

Además, se configurará el servicio SSH para permitir la administración remota del servidor de forma segura. Como mecanismo principal de autenticación se utilizará un sistema de criptografía de clave pública, compuesto por una clave privada almacenada únicamente en el equipo cliente y una clave pública registrada en el servidor. De esta forma, el acceso no dependerá exclusivamente del uso de una contraseña y se reducirá el riesgo asociado a intentos de autenticación mediante credenciales.

Durante los siguientes apartados se documentará paso a paso la configuración realizada, incluyendo los parámetros utilizados, los comandos de comprobación y las evidencias necesarias para verificar el correcto funcionamiento tanto de la red estática como del servicio SSH.

16.2 Configuración de dirección IP estática

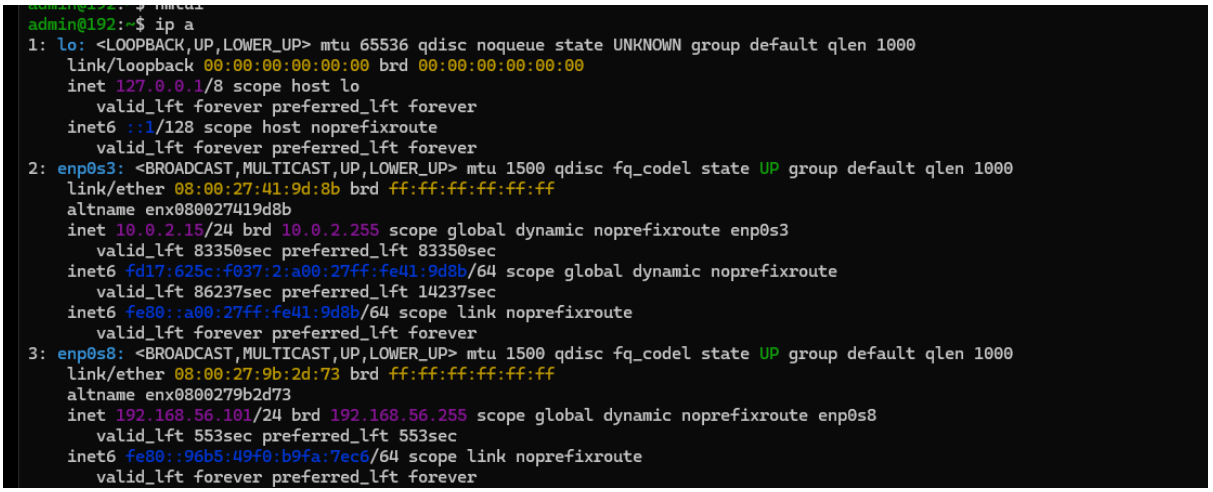
Para facilitar la administración del servidor y garantizar que pueda ser localizado siempre mediante la misma dirección dentro de la red de pruebas, se configuró una dirección IPv4 estática en la interfaz correspondiente al adaptador Host-Only de VirtualBox.

La red Host-Only permite la comunicación entre el equipo anfitrión y las máquinas virtuales conectadas a la misma red virtual, sin exponer directamente el servidor a redes externas. En este caso se utiliza la red 192.168.56.0/24, manteniendo separado este tráfico de la interfaz NAT utilizada para proporcionar acceso a Internet al servidor.

16.2.1 Identificación de la interfaz de red

Antes de modificar la configuración se verificaron las interfaces disponibles mediante el comando: **ip a**

Este comando permite visualizar las interfaces de red del sistema junto con las direcciones IP que tienen asignadas. De esta forma se puede identificar qué interfaz corresponde al adaptador Host-Only y evitar modificar por error la interfaz utilizada para la conexión NAT. En la captura proporcionada a continuación se pueden observar 3 interfaces de red: lo, enp0s3 y enp0s8, siendo esta última la interfaz correspondiente al adaptador Host-Only el cual vamos a configurar.



16.2.2 Configuración mediante nmtui

Para realizar la configuración se utilizó la herramienta nmtui a traves del siguiente comando: **sudo nmtui**

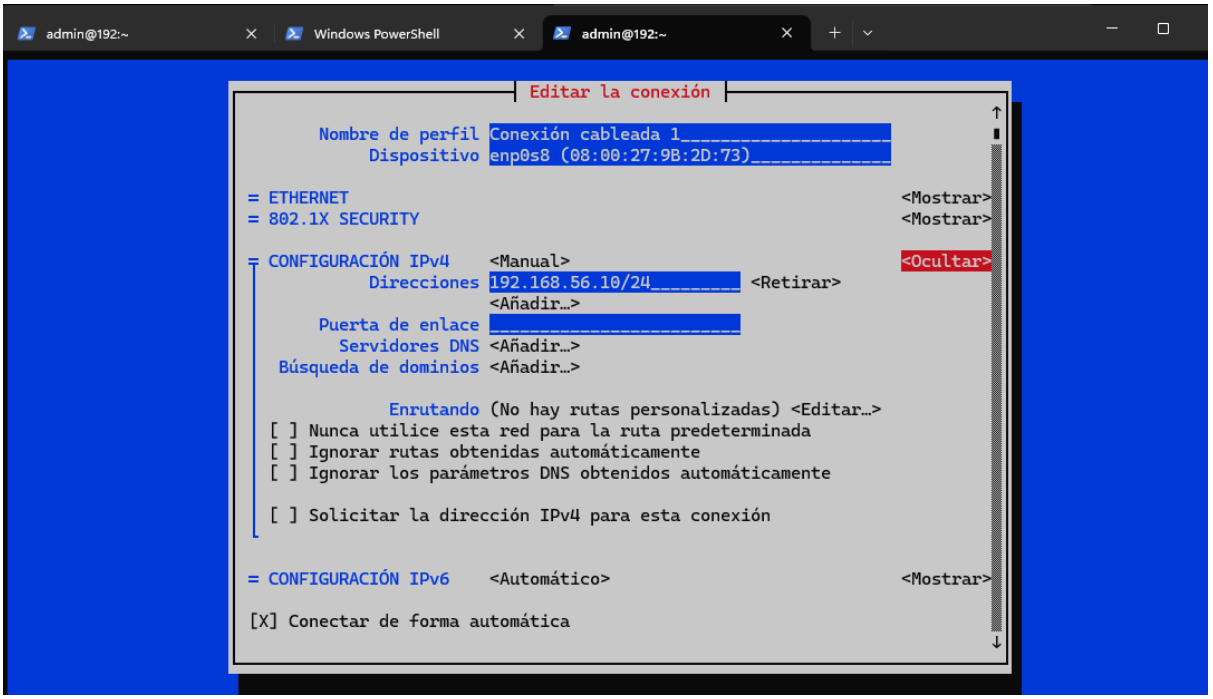
Esta herramienta proporciona una interfaz de texto para administrar las conexiones controladas por NetworkManager, permitiendo modificar los parámetros de red sin editar directamente los archivos de configuración.

Una vez iniciada la herramienta se seleccionó:

Editar una conexión → conexión correspondiente al adaptador Host-Only

Dentro de la configuración IPv4 se cambió el método de asignación a Manual y se estableció la siguiente dirección: **192.168.56.10/24**
La notación /24 hace referencia a la mascara de red: **255.255.255.0**
Por lo tanto el servidor queda ubicado en la red: **192.168.56.0/24**

En esta interfaz no es necesario configurar una puerta de enlace ni servidores DNS, ya que su función es únicamente proporcionar comunicación dentro de la red Host-Only. El acceso a Internet continúa realizándose mediante el adaptador NAT configurado de forma independiente.



16.2.3 Aplicación de la configuración

Una vez ingresados los parámetros se guardaron los cambios y se reactivó la conexión para que NetworkManager aplicara la nueva configuración.

Esto puede realizarse desde el propio nmtui, desactivando y volviendo a activar la conexión, o mediante los comandos:
sudo nmcli connection down “nombre_conexion”
sudo nmcli connection up “nombre_conexion”

Después de aplicar los cambios se volvió a comprobar la configuración mediante: **ip a**. La interfaz debe mostrar ahora la dirección estática configurada.

Al utilizar una configuración manual se evita depender de una dirección asignada dinámicamente por DHCP, lo que permite conocer de antemano la dirección del servidor y utilizarla posteriormente para servicios como SSH.

```
admin@10:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:41:9d:8b brd ff:ff:ff:ff:ff:ff
    altname enx080027419d8b
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
       valid_lft 86153sec preferred_lft 86153sec
    inet6 fd17:625c:f037:2:a00:27ff:fe41:9d8b/64 scope global dynamic noprefixroute
       valid_lft 86205sec preferred_lft 14205sec
    inet6 fe80::a00:27ff:fe41:9d8b/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:9b:2d:73 brd ff:ff:ff:ff:ff:ff
    altname enx0800279b2d73
    inet 192.168.56.10/24 brd 192.168.56.255 scope global noprefixroute enp0s8
       valid_lft forever preferred_lft forever
    inet6 fe80::96b5:49f0:b9fa:7ec6/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

16.2.4 Verificación de conectividad

Desde el equipo anfitrión se puede comprobar que existe comunicación con el servidor mediante el comando **ping**. Si la configuración es correcta el servidor responderá.

```
PS C:\Users\Usuario> ping 192.168.56.10

Haciendo ping a 192.168.56.10 con 32 bytes de datos:
Respuesta desde 192.168.56.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.56.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.56.10: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.56.10: bytes=32 tiempo<1m TTL=64

Estadísticas de ping para 192.168.56.10:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 0ms, Máximo = 0ms, Media = 0ms
PS C:\Users\Usuario>
```

También se puede verificar desde el propio servidor la tabla de rutas mediante: **ip route**. Esta comprobación permite verificar que la red 192.168.56.0/24 se encuentra

asociada a la interfaz Host-Only, mientras que la ruta utilizada para acceder a Internet continúa correspondiendo al adaptador NAT.

```
admin@192:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.10 metric 101
admin@192:~$
```

16.3 Configuración de SSH mediante clave pública y privada

En la primera etapa del proyecto se configuró y verificó el funcionamiento básico del servicio SSH, permitiendo acceder al servidor de forma remota mediante usuario y contraseña. En esta instancia se profundiza dicha configuración incorporando un mecanismo de autenticación mediante criptografía de clave pública, con el objetivo de mejorar la seguridad del acceso remoto.

Este sistema utiliza un par de claves relacionadas entre sí. La clave privada permanece almacenada únicamente en el dispositivo desde el cual se realizará la administración del servidor, mientras que la clave pública se copia al servidor y se asocia al usuario autorizado. Durante una conexión SSH, el servidor utiliza la clave pública para comprobar que el cliente posee la clave privada correspondiente, sin necesidad de transmitir dicha clave a través de la red.

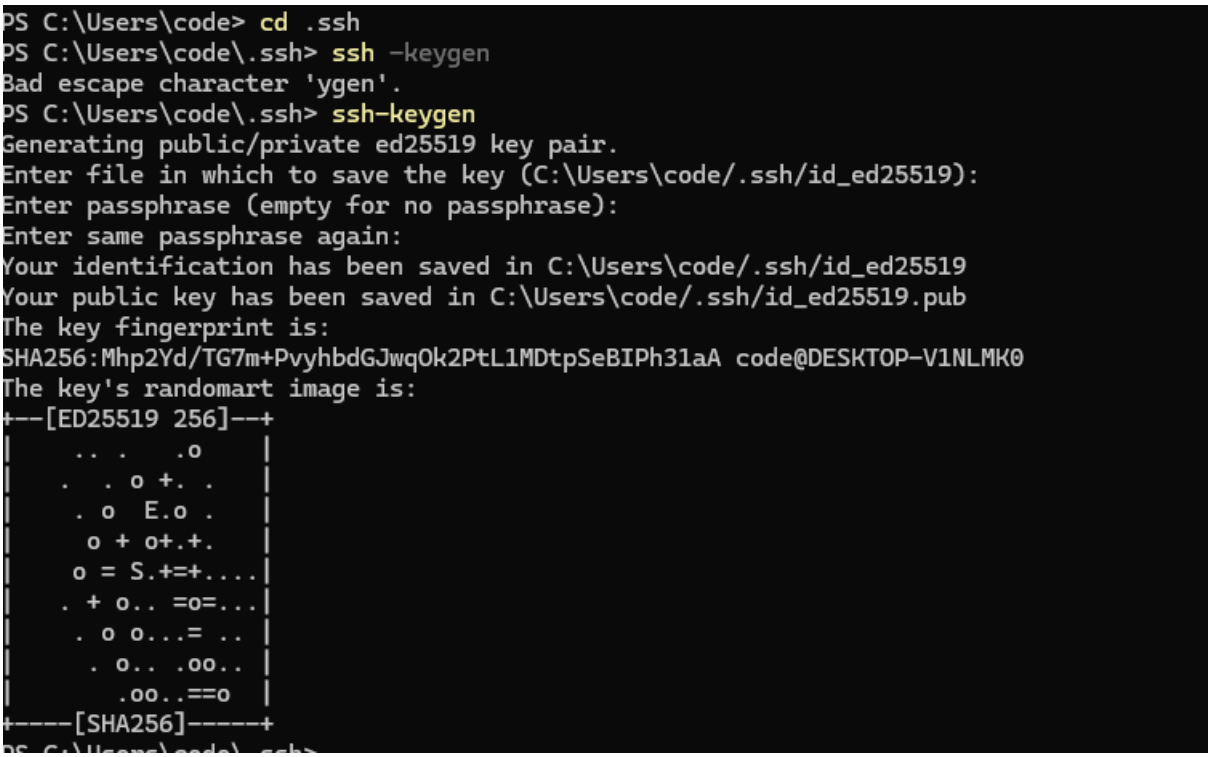
La principal ventaja de este mecanismo es que la clave privada nunca abandona el dispositivo del administrador. Además, puede protegerse mediante una contraseña propia, agregando una segunda medida de protección en caso de que el archivo de la clave sea obtenido por un tercero.

16.3.1 Generación del par de claves

El par de claves debe generarse desde el equipo cliente, es decir, desde el dispositivo utilizado para administrar remotamente el servidor.

Para generar las claves se utilizó el siguiente comando: **ssh-keygen**

Durante la ejecución se solicita indicar la ubicación donde se almacenarán las claves. Si se mantiene la ubicación propuesta por defecto, se generarán dos archivos dentro del directorio .ssh del usuario: id_ed25519 e id_ed25519.pub.
El archivo id_ed25519 corresponde a la clave privada y debe permanecer exclusivamente en el equipo cliente. Por otro lado, el archivo terminado en .pub corresponde a la clave pública, la cual debemos transferir al servidor.



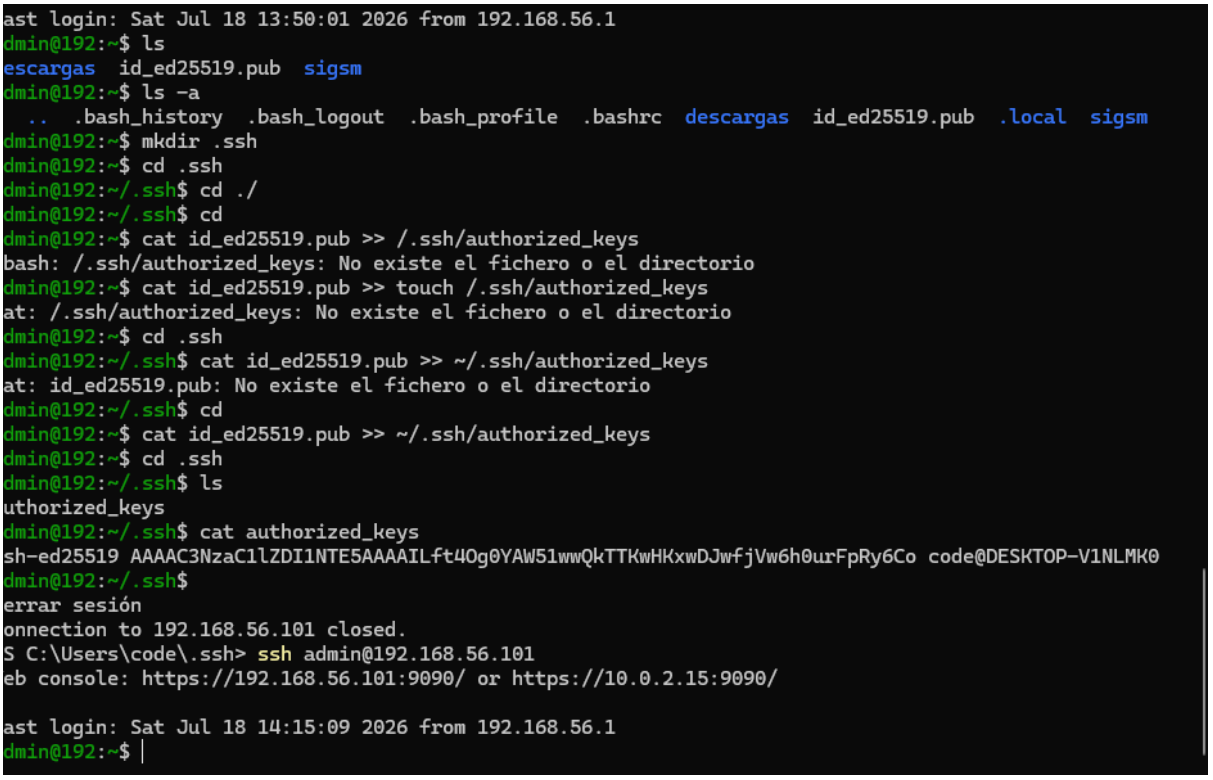
16.3.2 Incorporación de la clave pública al servidor

Para que el servidor reconozca al equipo cliente como autorizado, es necesario incorporar la clave pública generada anteriormente dentro del usuario que será utilizado para las conexiones SSH.

La clave debe almacenarse en: **~/.ssh/authorized_keys**
Para ello hacemos uso del siguiente comando: **scp "clave_publica" "usuario@IP:/carpeta/destino"**



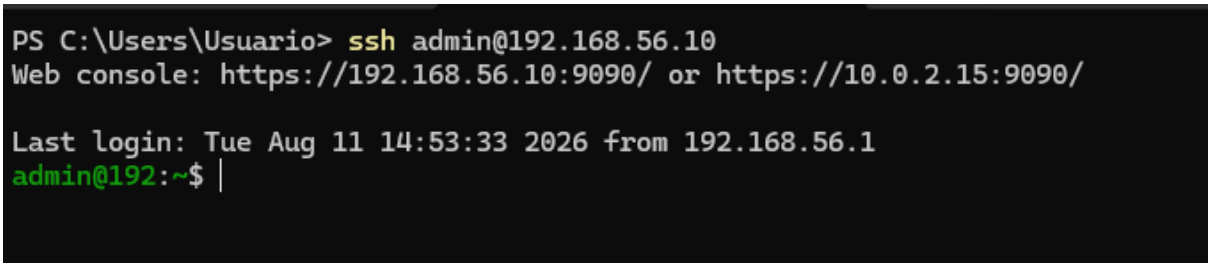
Una vez copiados los archivos se ingresa mediante SSH al servidor y se copia el contenido de nuestra clave pública en el destino correspondiente usando: **cat id_ed25519.pub >> ~/.ssh/authorized_keys**



16.3.3 Prueba de conexión mediante clave

Una vez incorporada la clave pública al servidor, se realiza una nueva conexión desde el equipo cliente mediante: **ssh usuario@IP**

Durante esta conexión el cliente SSH busca automáticamente la clave privada correspondiente y la utiliza para demostrar su identidad ante el servidor.
Si la autenticación se completa correctamente se obtiene acceso a la terminal remota sin utilizar la contraseña del usuario almacenada en el servidor.



16.3.4 Desactivación de autenticación mediante contraseña

Una vez comprobado que el acceso mediante clave pública funciona correctamente, se puede reforzar la configuración haciendo que SSH acepte únicamente este mecanismo de autenticación.

Para ello se edita: **sudo vim /etc/ssh/ssh_config** y se establecen las siguientes opciones:
PubkeyAuthentication yes (permite la autenticación mediante claves)
PasswordAuthentication no (impide iniciar sesión utilizando únicamente la contraseña del usuario)
PermitRootLogin no (impide iniciar sesión remotamente utilizando directamente la cuenta root)
Una vez modificado el archivo se procede a reiniciar el servicio mediante: **sudo systemctl restart sshd**.
Este cambio debe realizarse únicamente después de haber verificado que la autenticación mediante clave funciona correctamente, ya que de lo contrario podría perderse temporalmente el acceso remoto al servidor.

16.3.5 Resultados de la configuración

Después de aplicar esta configuración, el acceso remoto al servidor deja de depender únicamente de una combinación de usuario y contraseña. Para establecer una conexión SSH es necesario disponer de la clave privada correspondiente a una de las claves públicas autorizadas en el servidor.

La clave privada permanece bajo control del administrador y nunca se almacena ni se transmite al servidor. El servidor conserva únicamente la clave pública, por lo que su obtención no permite por sí sola autenticarse como usuario.

De esta manera se mejora la seguridad del servicio SSH previamente configurado y se establece un mecanismo más adecuado para la administración remota del servidor.

17. Configuración de grupos y permisos

17.1 Introducción

Como parte de la configuración y administración del servidor se implementó una estructura de grupos destinada a limitar las tareas administrativas que pueden realizar determinados usuarios del sistema. Esta configuración busca evitar que un usuario necesite disponer de privilegios generales de superusuario para realizar tareas específicas de mantenimiento, aplicando una separación de funciones de acuerdo con las responsabilidades asignadas.

Para ello se definieron dos grupos administrativos: uno encargado de la gestión del firewall y otro destinado a la administración de la base de datos. Los permisos correspondientes se configuraron mediante el archivo sudoers, permitiendo que los

integrantes de cada grupo ejecuten únicamente los comandos necesarios para su función. De esta forma se aplica el principio de mínimo privilegio y se reduce el acceso innecesario a otras funciones críticas del servidor.

17.2 Grupos administrativos definidos

17.2.1 Administrador del firewall

El grupo destinado a la administración del firewall tiene como función permitir que determinados usuarios puedan consultar y modificar la configuración de firewalld mediante la herramienta firewall-cmd.

Los integrantes de este grupo podrán realizar tareas como consultar las reglas activas, habilitar o deshabilitar puertos y servicios, aplicar configuraciones permanentes y recargar las reglas del firewall. Sin embargo, estos permisos no implican acceso administrativo sobre otras áreas del servidor.

La creación de este grupo permite delegar la gestión de la seguridad de red sin necesidad de proporcionar privilegios generales de superusuario.

17.2.2 Administrador de Base de Datos

El segundo grupo definido corresponde a los usuarios responsables de la administración de la base de datos. Su finalidad es permitir la ejecución de las herramientas necesarias para gestionar MariaDB y realizar operaciones relacionadas con el mantenimiento y respaldo de la información.

Entre las tareas previstas se encuentran el acceso al cliente de la base de datos y la generación de copias de seguridad mediante las herramientas correspondientes. Al

igual que en el caso anterior, los permisos se encuentran limitados exclusivamente a estos comandos, sin conceder acceso administrativo general al sistema.

Esta separación permite que la gestión de la base de datos pueda realizarse de forma independiente de la administración del firewall y de otras funciones críticas del servidor.

Durante las primeras etapas del proyecto, la base de datos se administró mediante MariaDB incluida en XAMPP, proporcionando un entorno sencillo para el desarrollo y las pruebas. Posteriormente, con la incorporación de Docker, se implementó MariaDB como un servicio independiente dentro de un contenedor, permitiendo administrar su ejecución, configuración y almacenamiento mediante Docker Compose. Esta nueva arquitectura también requirió adaptar los mecanismos de respaldo y los permisos de administración, aspectos que se desarrollan en los apartados correspondientes a la dockerización del sistema.

17.3 Configuración de permisos mediante sudoers

Una vez creados los grupos administrativos, fue necesario establecer qué comandos puede ejecutar cada uno con privilegios elevados. Para ello se utilizó el archivo sudoers, encargado de definir qué usuarios o grupos pueden utilizar sudo y bajo qué condiciones.

La configuración se realizó limitando los permisos únicamente a las herramientas necesarias para cada función, evitando otorgar acceso administrativo completo al servidor.

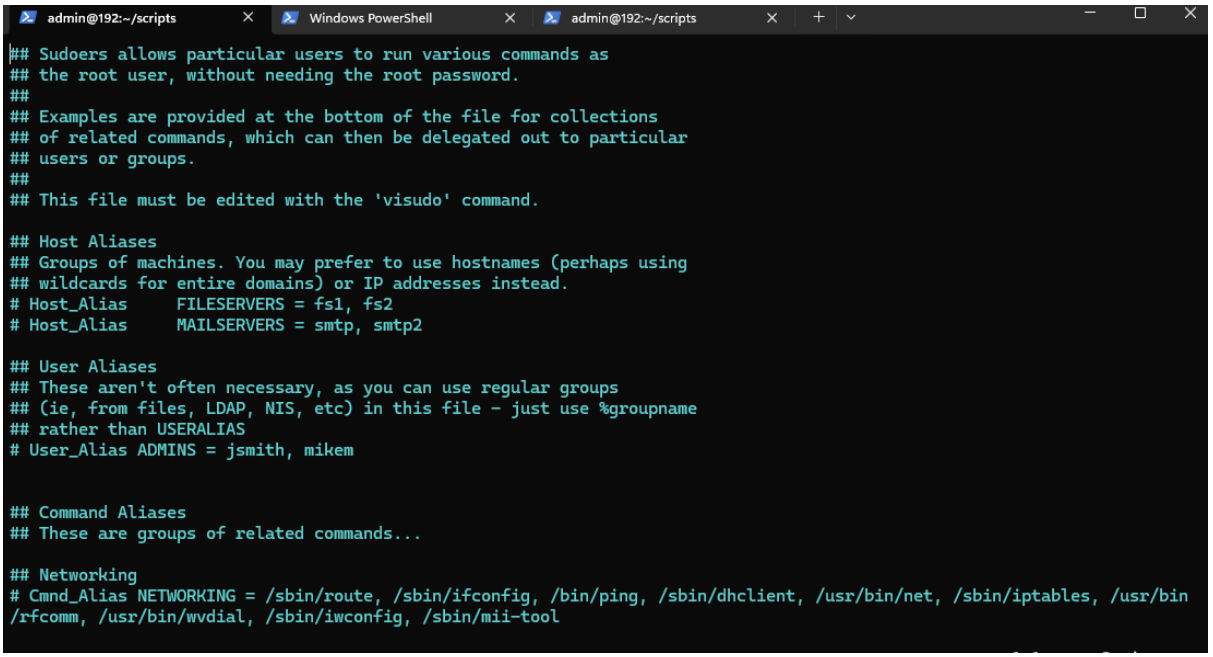
17.3.1 Acceso seguro al archivo

Para modificar la configuración se utilizó el comando: **sudo visudo**

El uso de visudo es preferible a editar directamente /etc/sudoers, ya que bloquea temporalmente el archivo durante la edición y comprueba su sintaxis antes de guardar los cambios. Esto reduce el riesgo de introducir errores que puedan impedir posteriormente el uso de sudo.

Antes de definir los permisos también se verificaron las rutas de los ejecutables que serían autorizados. Por ejemplo: **command -v firewall-cmd**

En el caso de las herramientas incluidas actualmente en XAMPP, sus ejecutables se encuentran dentro del directorio: **/opt/lampp/bin/**



17.3.2 Definición de comandos permitidos

Dentro del archivo se utilizaron alias de comandos mediante Cmnd_Alias, lo que permite agrupar de manera clara las herramientas correspondientes a cada función administrativa.

Para la administración del firewall se definió:

Cmnd_Alias FIREWALL_CMDS = /usr/bin/firewall-cmd, /home/admin/scripts/firewall-script.sh

Mientras que para la administración de la base de datos se establecieron las herramientas necesarias del entorno XAMPP:

Cmnd_Alias DATABASE_CMDS = /opt/lampp/bin/mysql, /opt/lampp/bin/mysqldump

De esta manera, los permisos otorgados no abarcan cualquier comando ejecutado como root, sino únicamente aquellos definidos expresamente.

```
# SIGSM config

# Firewall
Cmnd_Alias FIREWALL_CMDS = /usr/bin/firewall-cmd, /home/admin/scripts/firewall-script.sh

%fwadmins ALL=(root) FIREWALL_CMDS

# Base de datos
Cmnd_Alias DATABASE_CMDS = /opt/lampp/bin/mysql, /opt/lampp/bin/mysqldump

%dbadmins ALL=(root) DATABASE_CMDS
```

17.3.3 Aplicación de permisos a los grupos

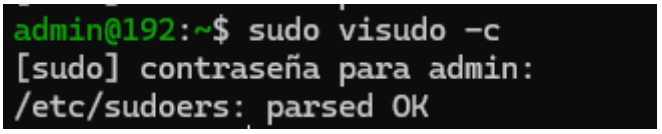
Finalmente, los conjuntos de comandos definidos fueron asociados a sus respectivos grupos mediante las siguientes reglas:

%fwadmins ALL=(root) FIREWALL_CMDS
%dbadmins ALL=(root) DATABASE_CMDS

El símbolo % indica que la regla se aplica a un grupo del sistema. De esta forma, cualquier usuario perteneciente a fwadmins puede ejecutar firewall-cmd mediante sudo, mientras que los integrantes de dbadmins pueden utilizar únicamente las herramientas autorizadas para la administración y respaldo de la base de datos.

Los usuarios de estos grupos no reciben permisos generales de superusuario ni acceso a los comandos asignados al otro grupo, manteniendo así la separación de funciones planteada inicialmente.

Para finalizar la configuración se realizó una comprobación de la sintaxis del archivo sudoers mediante el comando sudo visudo -c. Esta verificación permite detectar posibles errores antes de utilizar la configuración en el sistema. Al obtener un resultado correcto para los archivos de sudoers, se confirmó que las reglas definidas para los grupos administrativos fueron interpretadas correctamente y que la modificación no afecta el funcionamiento normal del comando sudo.



18. Docker

Para la instalación de Docker Engine en el servidor Fedora se utilizará el procedimiento oficial proporcionado por Docker para esta distribución. El proceso consiste en preparar el sistema para utilizar el repositorio oficial de Docker, agregar dicho repositorio e instalar los paquetes necesarios para ejecutar y administrar contenedores. La instalación se realiza mediante la terminal utilizando el gestor de paquetes DNF, lo que permite mantener Docker integrado con el sistema y facilitar posteriormente su actualización y mantenimiento.

18.1 Configuración del repositorio de Docker

En primer lugar, se comprueba que Docker no se encuentre instalado en nuestro sistema y de estarlo se procede a eliminar usando el siguiente comando:

```
sudo dnf remove docker \
              docker-client \
              docker-client-latest \
              docker-common \
              docker-latest \
              docker-latest-logrotate \
              docker-logrotate \
              docker-selinux \
              docker-engine-selinux \
              docker-engine
```

Una vez ejecutado ya se puede comenzar la instalación. Para ello se agrega al sistema el repositorio oficial de Docker. Esto permite que Fedora pueda descargar e instalar Docker Engine directamente desde los servidores oficiales y facilita también futuras actualizaciones. El comando es el siguiente:

sudo dnf config-manager addrepo --from-repofile https://download.docker.com/linux/fedora/docker-ce.repo

Una vez ejecutado se debería ver algo parecido a la siguiente captura:

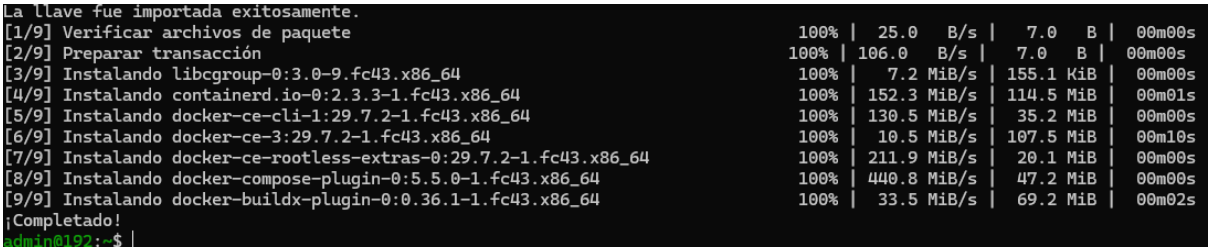


18.2 Instalación de Docker Engine

Una vez agregado el repositorio, se procede a instalar Docker Engine junto con las herramientas necesarias para su funcionamiento. También se instalan los complementos de Docker Compose y Buildx.

sudo dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

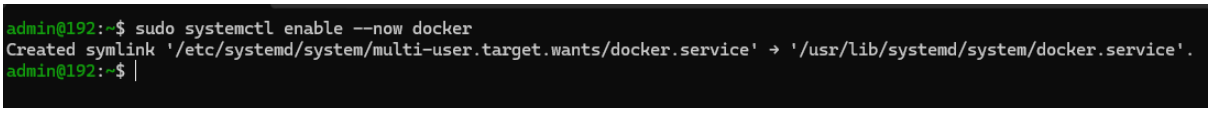
Durante la instalación, el sistema solicitará confirmación para descargar los paquetes y puede solicitar la aceptación de la clave GPG del repositorio de Docker. Se aceptan las opciones correspondientes para continuar con la instalación.



18.3 Inicio y habilitación del servicio

Después de completar la instalación, se habilita el servicio de Docker y se inicia inmediatamente mediante systemctl. Al utilizar la opción enable, Docker también quedará configurado para iniciarse automáticamente junto con el sistema operativo.

sudo systemctl enable --now docker



18.4 Comprobación de la instalación

Finalmente, se verifica que Docker Engine se encuentre correctamente instalado y funcionando mediante la ejecución del contenedor de prueba hello-world.

sudo docker run hello-world

Este comando descarga una imagen de prueba desde Docker Hub y crea un contenedor temporal. Si la instalación y el servicio funcionan correctamente, Docker mostrará el mensaje “Hello from Docker!”, confirmando que el entorno está listo para utilizar contenedores.

```
admin@192:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5dd0d3e6e255913fc30f90b9f2b1d359cc2cbdb48090cc4b65f1676e203243cc
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

19. Política y gestión de respaldos

19.1 Introducción

La realización de respaldos constituye una medida fundamental para garantizar la disponibilidad y recuperación de la información almacenada por el sistema S.I.G.S.M. Ante posibles fallos del servidor, errores humanos, corrupción de archivos o pérdida accidental de información, disponer de copias de seguridad permite recuperar los datos necesarios para restablecer el funcionamiento del sistema.

Para el proyecto se definió una política de respaldos que contempla tanto copias locales como remotas. Los respaldos locales permiten disponer de una copia de recuperación accesible directamente desde el servidor, mientras que los respaldos remotos mantienen una copia adicional en otro equipo, reduciendo el riesgo de pérdida de información en caso de que el servidor principal resulte afectado.

Los procesos de respaldo se encuentran automatizados mediante cron, permitiendo que las copias se realicen periódicamente sin necesidad de intervención manual. Se seleccionaron horarios de baja actividad con el objetivo de reducir el impacto de estas tareas sobre el funcionamiento habitual del sistema.

19.2 Información incluida en los respaldos

Los respaldos definidos para S.I.G.S.M. contemplan la información necesaria para poder recuperar el funcionamiento del sistema ante una pérdida de datos. Para ello se consideran dos componentes principales: la base de datos, donde se almacena la información estructurada utilizada por la aplicación, y los documentos gestionados por el módulo de documentación.

Ambos elementos deben respaldarse de forma conjunta, ya que la recuperación de únicamente uno de ellos podría dejar al sistema en un estado incompleto. Por ejemplo, restaurar la base de datos sin recuperar los archivos asociados permitiría conservar sus registros, pero los documentos referenciados por estos podrían no encontrarse disponibles.

19.2.1 Respaldo de la base de datos

La base de datos contiene la información utilizada por los diferentes módulos del sistema, incluyendo datos relacionados con funcionarios, roles, categorías,

documentos, encuestas y traslados, entre otros elementos necesarios para el funcionamiento de la aplicación.

Para generar una copia de seguridad se utiliza la herramienta mysqldump incluida en la instalación de MariaDB utilizada mediante XAMPP. Esta herramienta permite exportar la estructura de la base de datos y sus registros a un archivo SQL, que posteriormente puede utilizarse para reconstruir la información en caso de ser necesario.

Durante el proceso de respaldo se genera inicialmente un archivo .sql temporal. Posteriormente, este archivo se incluye dentro del respaldo comprimido, reduciendo el espacio ocupado y facilitando su almacenamiento o transferencia hacia otro servidor.

De esta forma, en caso de pérdida o corrupción de la base de datos principal, se dispone de una copia que permite recuperar tanto la estructura de las tablas como los datos almacenados hasta el momento en que se realizó el respaldo.

19.2.2 Respaldo de documentos de la aplicación

Además de la información almacenada en la base de datos, el módulo de documentación de S.I.G.S.M. administra archivos que son cargados al servidor y posteriormente puestos a disposición de los pacientes mediante la aplicación.

Estos documentos se almacenan físicamente dentro del sistema de archivos del servidor, mientras que la base de datos conserva la información asociada a cada uno de ellos, como su título, categoría, descripción y ubicación. Por este motivo, el

respaldo de la base de datos por sí solo no resulta suficiente para recuperar completamente este módulo.

La política de respaldos contempla también la copia de la carpeta donde se encuentran almacenados dichos documentos. Esta carpeta se incorpora al mismo conjunto de respaldo que la base de datos y posteriormente se comprime utilizando tar junto con gzip.

De esta manera, cada respaldo automático contiene tanto una copia de la base de datos como de los archivos utilizados por la aplicación, permitiendo conservar una versión coherente de la información necesaria para una futura recuperación del sistema.

19.3 Política de respaldos

Con el objetivo de mantener copias actualizadas de la información sin generar una carga innecesaria sobre el servidor, se establecieron diferentes frecuencias para los respaldos locales y remotos. La programación de estas tareas se realiza mediante cron, permitiendo ejecutar los procesos automáticamente en los días y horarios definidos.

La política adoptada prioriza la realización frecuente de copias locales, debido a que su generación no requiere transferir información a través de la red. Los respaldos remotos se realizan con menor frecuencia, ya que implican además la transferencia de los archivos hacia otro equipo.

Los horarios seleccionados corresponden a períodos de baja actividad, buscando disminuir el impacto de las operaciones de compresión, acceso a la base de datos y transferencia de archivos sobre el funcionamiento habitual del sistema.

19.3.1 Respaldo local diario

Se estableció la realización de un respaldo local todos los días a las 04:00 horas. Esta frecuencia permite conservar una copia reciente de la base de datos y de los documentos almacenados por la aplicación, limitando la cantidad de información que podría perderse ante un fallo del servidor.

El horario de las 04:00 fue seleccionado por corresponder a un período en el que se espera una baja utilización del sistema. De esta manera, operaciones como la exportación de la base de datos y la compresión de los archivos pueden ejecutarse reduciendo su posible impacto sobre los usuarios.

La expresión utilizada para programar esta tarea mediante cron es:

```
0 4 * * * /home/admin/scripts/cron/local.sh
```

19.3.2 Respaldo remoto semanal

Además de las copias locales, se estableció un respaldo remoto durante el fin de semana. Su finalidad es mantener una copia de la información fuera del servidor principal, permitiendo disponer de una alternativa de recuperación en caso de que las copias almacenadas localmente también resulten afectadas.

El respaldo remoto se realizará los domingos a las 05:00 horas. Se seleccionó una frecuencia semanal debido a que la información ya cuenta con respaldos locales

diarios y la copia remota requiere, además, transferir el archivo generado hacia otro equipo mediante la red.

La ejecución durante la madrugada y en fin de semana busca reducir tanto el uso de recursos del servidor como el tráfico generado por la transferencia del respaldo.

La expresión utilizada en cron es:

```
0 5 * * 0 /home/admin/scripts/cron/remote.sh
```

19.4 Ubicación de los respaldos

Los respaldos generados por el sistema se almacenan en ubicaciones diferentes según se trate de una copia local o remota. Esta separación permite disponer de una copia de recuperación accesible directamente desde el servidor y, al mismo tiempo, mantener una segunda copia en otro equipo.

La ubicación de los respaldos fue definida de forma que los archivos puedan identificarse y localizarse fácilmente. Los nombres de los archivos incluyen la fecha y hora de creación, permitiendo diferenciar las distintas versiones generadas a lo largo del tiempo.

19.4.1 Almacenamiento local

Los respaldos locales se almacenan dentro del servidor principal en el directorio: /home/admin/Backups

En esta ubicación se guardan los archivos comprimidos generados durante los procesos de respaldo. Cada archivo utiliza un nombre identificativo acompañado de la fecha y hora de creación, por ejemplo: sigsmDB_30-08-2026_04-00.tar.gz

El uso de un directorio específico permite mantener las copias de seguridad separadas del resto de los archivos utilizados por la aplicación y facilita su administración posterior.

Los respaldos almacenados localmente constituyen la primera alternativa de recuperación ante una pérdida o modificación accidental de información, ya que pueden ser utilizados directamente desde el propio servidor sin necesidad de acceder a otro equipo.

19.4.2 Almacenamiento remoto

Además de las copias locales, se mantiene un respaldo en un servidor remoto conectado a la red. Los archivos son transferidos mediante scp, utilizando el servicio SSH previamente configurado.

Para el entorno de pruebas se utiliza como destino remoto el servidor identificado por la dirección: 192.168.56.102

Los respaldos son almacenados en el directorio /home/admin/Backups del equipo remoto.

La transferencia se realiza utilizando autenticación mediante claves SSH, evitando la necesidad de ingresar manualmente una contraseña durante la ejecución automática de los respaldos.

Mantener una copia en otro equipo permite conservar la información incluso en situaciones donde el servidor principal o su almacenamiento local resulten

afectados. De esta forma, el respaldo remoto actúa como una segunda medida de protección complementaria a las copias realizadas diariamente en el servidor principal.

20. Dockerización y despliegue de S.I.G.S.M.

20.1. Introducción a la dockerización

Como parte de la preparación del entorno de despliegue del sistema S.I.G.S.M., se implementó una arquitectura basada en contenedores mediante Docker. Esta tecnología permite ejecutar los diferentes componentes de la aplicación en entornos aislados, con sus propias dependencias y configuraciones, facilitando su instalación, administración y mantenimiento.

Durante las primeras etapas del proyecto se utilizó XAMPP como entorno de desarrollo y pruebas, proporcionando los servicios de Apache, PHP y MariaDB necesarios para ejecutar el backend y administrar la base de datos. Sin embargo, con el avance del desarrollo surgió la necesidad de disponer de un mecanismo de despliegue más organizado y reproducible, que permitiera ejecutar la aplicación completa sin depender de la instalación y configuración manual de cada componente en el sistema operativo anfitrión.

Para ello, se desarrolló una configuración basada en Docker Compose, mediante la cual se administran los servicios correspondientes al frontend, backend y base de datos, además de determinados procesos auxiliares utilizados durante la inicialización del sistema. Esta organización permite que cada componente disponga de un entorno de ejecución independiente y que los servicios se comuniquen mediante una red interna de Docker.

La implementación se realizó sobre el servidor Fedora Server 43 configurado previamente en una máquina virtual mediante VirtualBox. La instalación de Docker Engine y la comprobación de su funcionamiento se encuentran documentadas en el apartado 18 del presente documento.

En los siguientes apartados se describirán las razones que motivaron la elección de esta tecnología, la arquitectura de contenedores implementada, los archivos utilizados para su configuración y el procedimiento necesario para desplegar la aplicación. También se abordarán aspectos relacionados con la persistencia de datos, la administración de los servicios, los respaldos y las medidas de seguridad aplicadas al entorno.

20.2. Justificación de la utilización de Docker

La elección de Docker como herramienta para el despliegue de S.I.G.S.M. responde a la necesidad de disponer de un entorno de ejecución organizado, reproducible y sencillo de administrar. Al tratarse de una aplicación web compuesta por diferentes tecnologías, resulta conveniente separar sus componentes y establecer una configuración que permita instalarlos y ejecutarlos de forma conjunta.

La utilización de contenedores facilita esta organización, permitiendo definir las dependencias y configuraciones necesarias para cada servicio sin tener que instalarlas directamente en el sistema operativo del servidor.

20.2.1. Transición del entorno XAMPP a contenedores

Durante las primeras etapas del proyecto se utilizó XAMPP como herramienta principal para ejecutar los servicios de Apache, PHP y MariaDB. Esta solución

permitió preparar rápidamente un entorno funcional, realizar pruebas de los endpoints del backend y trabajar con la base de datos sin necesidad de configurar cada servicio de forma independiente.

Sin embargo, XAMPP presenta ciertas limitaciones cuando se pretende utilizar el mismo entorno para desplegar una aplicación completa. En particular, su utilización requiere administrar conjuntamente diferentes componentes y mantener sus configuraciones dentro de una misma instalación. Además, el frontend desarrollado con React necesita herramientas adicionales para su construcción y ejecución, que no forman parte de XAMPP.

Con la incorporación de Docker se optó por separar los componentes del sistema en diferentes contenedores, cada uno con las tecnologías necesarias para cumplir su función. De esta manera, el frontend, el backend y la base de datos pueden ejecutarse de forma independiente, manteniendo la comunicación necesaria para el funcionamiento de la aplicación.

Esta transición no implicó abandonar XAMPP durante el desarrollo, sino incorporar una alternativa más adecuada para la distribución y puesta en funcionamiento del sistema. XAMPP continúa siendo una herramienta útil para realizar determinadas pruebas, mientras que Docker permite preparar un entorno de despliegue completo mediante una configuración previamente definida.

20.2.2. Ventajas de Docker para el despliegue del sistema

Una de las principales ventajas de Docker es la posibilidad de reproducir el entorno de ejecución de la aplicación en diferentes equipos. Mediante la construcción de imágenes se incorporan las dependencias y configuraciones necesarias para cada

servicio, reduciendo las diferencias que pueden surgir al instalar manualmente las tecnologías utilizadas.

En el caso de S.I.G.S.M., esto permite ejecutar la aplicación en un servidor que disponga de Docker sin necesidad de instalar individualmente PHP, Apache, Nginx, Node.js o MariaDB en el sistema operativo anfitrión. Los componentes necesarios se incorporan en las imágenes correspondientes y se ejecutan dentro de sus respectivos contenedores.

Otra ventaja importante es la separación de los servicios. Al disponer de contenedores independientes, es posible administrar, reiniciar o reconstruir determinados componentes sin tener que reinstalar el resto de la aplicación. Por ejemplo, una modificación en el frontend puede incorporarse mediante la reconstrucción de su imagen sin necesidad de modificar directamente la instalación de MariaDB.

Docker también facilita el mantenimiento y la actualización del sistema, ya que permite reconstruir las imágenes utilizando nuevas versiones del código o cambios en las configuraciones. En el proyecto, los archivos utilizados para construir las imágenes se encuentran versionados en GitHub, lo que permite mantener un registro de las modificaciones realizadas.

Por último, la utilización de volúmenes persistentes permite separar los datos de los contenedores que ejecutan los servicios. Esta característica resulta especialmente importante para S.I.G.S.M., dado que tanto la base de datos como los documentos cargados por los funcionarios deben conservarse incluso cuando los contenedores sean detenidos o reconstruidos.

20.2.3. Utilización de Docker Compose

Para administrar los contenedores del proyecto se seleccionó Docker Compose, una herramienta que permite definir y ejecutar aplicaciones compuestas por múltiples servicios mediante un archivo de configuración en formato YAML.

En lugar de crear y configurar manualmente cada contenedor mediante comandos individuales, Docker Compose permite establecer en un único archivo las imágenes utilizadas, los servicios que deben ejecutarse, las variables de entorno, los volúmenes, las redes y las dependencias existentes entre los componentes.

En S.I.G.S.M., esta configuración se encuentra definida en el archivo compose.yaml, ubicado dentro del directorio prototipos/docker del repositorio. Mediante este archivo se administran los servicios principales de frontend, backend y base de datos, además de los servicios auxiliares destinados a la creación del administrador inicial y la carga opcional de datos de demostración.

La utilización de Docker Compose simplifica considerablemente el procedimiento de instalación, ya que permite construir las imágenes e iniciar los servicios necesarios mediante un único comando. Asimismo, facilita las tareas de administración, permitiendo consultar el estado de los contenedores, revisar sus registros, detenerlos o reconstruirlos cuando resulte necesario.

Otra característica relevante es la posibilidad de establecer dependencias entre servicios. Esto permite controlar determinados aspectos del orden de inicialización, como comprobar que MariaDB se encuentre disponible antes de ejecutar los procesos que requieren acceso a la base de datos.

Por estos motivos, Docker Compose fue seleccionado como herramienta principal para organizar el despliegue de S.I.G.S.M., proporcionando un mecanismo centralizado para administrar sus componentes y facilitando la instalación de la aplicación en diferentes entornos de prueba.

20.3. Arquitectura de contenedores

La arquitectura de contenedores de S.I.G.S.M. fue diseñada para separar los diferentes componentes de la aplicación según su función, permitiendo administrar sus dependencias y configuraciones de manera independiente.

Para ello, se implementó una estructura compuesta por tres servicios principales: frontend, backend y base de datos. Adicionalmente, se incorporaron dos servicios auxiliares destinados a realizar determinadas tareas durante la inicialización del sistema.

Todos estos servicios se encuentran definidos en el archivo compose.yaml y se comunican mediante una red virtual creada por Docker Compose.

20.3.1. Organización general de los servicios

La aplicación se encuentra distribuida en cinco servicios, cada uno encargado de una función específica dentro del sistema.

Servicio	Tecnología	Función
frontend	React y Nginx	Interfaz web y recepción de solicitudes HTTP

backend	PHP 8.2 y Apache	Procesamiento de solicitudes y ejecución de la API
db	MariaDB 11.4	Almacenamiento y administración de la información
admin-init	PHP	Creación del administrador inicial
seed-demo	MariaDB	Carga opcional de información de demostración

Los servicios frontend, backend y db permanecen en ejecución para garantizar el funcionamiento de la aplicación. Por su parte, admin-init y seed-demo están diseñados para ejecutar tareas puntuales y finalizar una vez completadas.

Esta organización permite separar las responsabilidades de cada componente, facilitando su mantenimiento y evitando que todos los servicios dependan de una única instalación de software.

20.3.2. Frontend: React y Nginx

El servicio frontend es el encargado de proporcionar la interfaz web de S.I.G.S.M. a los usuarios. Para ello, utiliza React como tecnología de desarrollo de la interfaz y Nginx como servidor HTTP encargado de distribuir los archivos generados durante la construcción de la aplicación.

Durante la creación de la imagen Docker, el código del frontend se obtiene desde el repositorio de GitHub y se ejecuta el proceso de compilación mediante Node.js y Vite. Como resultado, se generan los archivos estáticos necesarios para ejecutar la aplicación, los cuales posteriormente se incorporan a una imagen de Nginx.

De esta manera, Node.js no necesita permanecer ejecutándose en el contenedor final, ya que Nginx se encarga de servir los archivos de la aplicación al navegador.

Además de proporcionar el frontend, Nginx funciona como proxy inverso para las solicitudes dirigidas a la API. Cuando el navegador realiza una petición cuya ruta comienza con /api/, Nginx la redirige al servicio backend a través de la red de Docker.

Este contenedor es el único servicio de la aplicación que publica un puerto en el equipo anfitrión. Por defecto, se utiliza el puerto 8080, aunque este valor puede modificarse mediante las variables de entorno.

20.3.3. Backend: PHP y Apache

El servicio backend contiene la API REST desarrollada en PHP 8.2 y utiliza Apache HTTP Server para recibir y procesar las solicitudes provenientes del frontend.

Su función principal es ejecutar la lógica de negocio correspondiente a los diferentes módulos de S.I.G.S.M., incluyendo la autenticación de funcionarios, gestión documental, encuestas, administración de vehículos y solicitudes de traslado.

Durante la construcción de la imagen se incorpora el código PHP disponible en GitHub, junto con las extensiones necesarias para su funcionamiento y los archivos de configuración de Apache.

El servidor Apache utiliza como directorio público /var/www/api/public, desde el cual se atienden las solicitudes de la aplicación. También se configura el acceso mediante el prefijo /api, permitiendo organizar los endpoints sin necesidad de exponer directamente toda la estructura interna del código.

Para acceder a la información almacenada, el backend establece una conexión con MariaDB utilizando el nombre del servicio db como dirección del servidor de base de datos.

A diferencia del frontend, el backend no publica directamente su puerto en el equipo anfitrión. Las solicitudes de los usuarios llegan a través de Nginx, mientras que la comunicación entre ambos contenedores se realiza mediante la red interna definida en Docker Compose.

20.3.4. Base de datos: MariaDB

El servicio db utiliza MariaDB 11.4 como sistema gestor de bases de datos relacional. Su función es almacenar y administrar la información utilizada por los diferentes módulos de S.I.G.S.M., incluyendo funcionarios, roles, categorías, documentos, encuestas, vehículos y traslados.

La imagen utilizada se construye a partir de la imagen oficial de MariaDB, incorporando los scripts SQL necesarios para crear la estructura inicial de la base de datos.

Durante una instalación nueva, estos scripts se ejecutan automáticamente y permiten generar las tablas, relaciones, restricciones y demás elementos necesarios para el funcionamiento del sistema.

La base de datos se identifica mediante el nombre sigsm y dispone de credenciales configuradas a través de variables de entorno, evitando que estas deban escribirse directamente en el código de la aplicación.

Para conservar la información almacenada, el servicio utiliza un volumen persistente denominado db_data, asociado al directorio /var/lib/mysql del contenedor. Esto permite mantener los datos aunque el contenedor sea detenido, eliminado o reconstruido, siempre que el volumen no sea eliminado.

Adicionalmente, se implementó una comprobación de estado mediante healthcheck, que permite verificar que MariaDB se encuentre disponible y que la estructura requerida haya sido inicializada correctamente antes de continuar con determinados servicios.

Al igual que el backend, MariaDB no publica su puerto directamente en el equipo anfitrión, por lo que las conexiones se realizan a través de la red definida para los contenedores.

20.3.5. Servicios auxiliares: admin-init y seed-demo

Además de los tres servicios principales, se incorporaron dos servicios auxiliares destinados a facilitar la configuración inicial del sistema y la preparación de entornos de demostración.

El primero de ellos, denominado admin-init, tiene como finalidad crear automáticamente una cuenta administrativa durante la instalación inicial de S.I.G.S.M.

Este servicio utiliza la misma imagen del backend, pero ejecuta un script PHP específico encargado de comprobar la existencia del administrador, generar el hash de su contraseña y asignarle el rol correspondiente.

Las credenciales iniciales se obtienen de las variables de entorno configuradas antes de iniciar el despliegue. El proceso se ejecuta una vez que MariaDB se encuentra disponible y finaliza después de completar las operaciones necesarias.

Por otra parte, el servicio seed-demo permite incorporar información ficticia a la base de datos con fines de prueba y demostración.

Este servicio utiliza un script SQL que genera registros de ejemplo, incluyendo funcionarios, vehículos, traslados, categorías y encuestas, entre otros datos necesarios para probar las funcionalidades del sistema.

A diferencia de los servicios principales, seed-demo no se ejecuta durante una instalación normal. Su activación se realiza mediante el perfil demo de Docker Compose, permitiendo seleccionar si se desea preparar un entorno con información de demostración.

Ambos servicios fueron diseñados para ejecutar tareas puntuales y finalizar correctamente, sin necesidad de permanecer activos durante el funcionamiento habitual de la aplicación.

20.3.6. Comunicación y red interna entre contenedores

Para permitir la comunicación entre los diferentes componentes se configuró una red de Docker denominada internal, utilizando el controlador de red bridge.

Esta red permite que los contenedores se comuniquen entre sí mediante sus nombres de servicio, sin necesidad de establecer direcciones IP estáticas para cada uno.

El funcionamiento general comienza cuando un usuario accede a S.I.G.S.M. desde su navegador. La solicitud llega al contenedor frontend, donde Nginx se encarga de proporcionar los archivos de la aplicación.

Cuando es necesario consultar o modificar información, el frontend realiza una petición HTTP a la API. Nginx recibe dicha petición y la redirige al contenedor backend, donde Apache ejecuta el código PHP correspondiente.

Si la operación requiere acceder a la base de datos, el backend establece una conexión con MariaDB mediante el nombre db y el puerto 3306. Una vez procesada la solicitud, la respuesta se devuelve al frontend para que pueda presentar los resultados al usuario.

La configuración permite que únicamente Nginx publique un puerto hacia el equipo anfitrión, mientras que Apache y MariaDB permanecen accesibles a través de la red de contenedores.

También se definieron dependencias de inicialización mediante la propiedad depends_on de Docker Compose. En particular, el backend requiere que MariaDB se encuentre en estado saludable y que el proceso de creación del administrador inicial haya finalizado correctamente antes de iniciarse.

Esta configuración contribuye a organizar el inicio de los servicios y reducir errores relacionados con la disponibilidad de la base de datos.

En conjunto, la arquitectura implementada proporciona una separación clara entre la interfaz, la lógica de negocio y el almacenamiento de información, permitiendo

administrar sus componentes de forma organizada y facilitando futuras tareas de mantenimiento y actualización.

20.4. Estructura y archivos de configuración

Para organizar el despliegue de S.I.G.S.M. se creó un directorio específico dentro del repositorio de GitHub, destinado a almacenar los archivos necesarios para construir y administrar los contenedores de la aplicación.

Esta organización permite mantener separadas las configuraciones de Docker del código fuente utilizado durante el desarrollo, facilitando su mantenimiento y permitiendo realizar la instalación del sistema a partir de un conjunto reducido de archivos.

20.4.1. Organización del directorio Docker

Los archivos utilizados para el despliegue se encuentran dentro del directorio prototipos/docker del repositorio principal de S.I.G.S.M.

En esta ubicación se agrupan los archivos de configuración general y tres directorios correspondientes a los componentes principales de la aplicación: frontend, backend y base de datos.

La estructura se organiza de la siguiente manera:

Archivo o directorio	Función
compose.yaml	Define los servicios, redes, volúmenes y configuraciones generales de Docker Compose.
.env.example	Proporciona una plantilla de las variables de entorno necesarias.

.env	Contiene las variables y credenciales utilizadas en la instalación.
.gitignore	Evita incorporar archivos de configuración sensibles al repositorio.
README.md	Documenta el procedimiento de instalación, administración y mantenimiento.
frontend/Dockerfile	Define la construcción de la imagen del frontend.
frontend/nginx.conf	Contiene la configuración del servidor Nginx.
backend/Dockerfile	Define la construcción de la imagen del backend.
backend/apache.conf	Contiene la configuración de Apache HTTP Server.
backend/php.ini	Establece parámetros de ejecución de PHP.
backend/bootstrap-admin.php	Contiene la lógica de creación del administrador inicial.
database/Dockerfile	Define la imagen de MariaDB e incorpora los scripts SQL.

El archivo .env se genera durante la instalación a partir de la plantilla .env.example y no se incorpora al repositorio, debido a que contiene credenciales utilizadas por los servicios.

Esta distribución permite identificar fácilmente qué archivos corresponden a cada componente y modificar sus configuraciones sin afectar directamente a los demás servicios.

20.4.2. Construcción de imágenes mediante Dockerfiles

Para construir las imágenes utilizadas por S.I.G.S.M. se crearon tres Dockerfiles independientes, correspondientes al frontend, backend y base de datos.

Estos archivos contienen las instrucciones necesarias para preparar el entorno de ejecución de cada componente, incluyendo la selección de imágenes base, instalación de dependencias y copia de archivos de configuración.

Una característica importante de la implementación es que los Dockerfiles obtienen el código fuente directamente desde el repositorio de GitHub durante el proceso de construcción. Para ello se utiliza Git y la funcionalidad sparse-checkout, que permite descargar únicamente los directorios necesarios para cada componente.

De esta manera, el servidor encargado del despliegue no necesita mantener una copia completa del código fuente de la aplicación. Las imágenes se construyen utilizando el código publicado en la rama main del repositorio.

Imagen del frontend

La construcción del frontend se realiza mediante varias etapas. Inicialmente se obtiene el código fuente desde GitHub y posteriormente se utiliza una imagen de Node.js 22 para instalar las dependencias mediante npm ci y generar los archivos de producción con npm run build.

Una vez finalizada la compilación, los archivos generados se incorporan a una imagen basada en Nginx 1.28, que será la encargada de servir la aplicación.

Este procedimiento permite que la imagen final contenga únicamente los componentes necesarios para ejecutar el frontend, sin mantener Node.js ni las herramientas de compilación como procesos activos.

Imagen del backend

El backend utiliza como base una imagen de PHP 8.2 con Apache, basada en Debian Bookworm.

Durante su construcción se instalan las extensiones pdo_mysql y mbstring, necesarias para la comunicación con MariaDB y el procesamiento de cadenas de caracteres. También se habilitan los módulos rewrite y headers de Apache.

Posteriormente, se incorpora el código de la API desde GitHub y se copian los archivos de configuración correspondientes a Apache y PHP.

Además, se prepara el directorio /var/www/api/storage, utilizado para almacenar los archivos gestionados por la aplicación, estableciendo los permisos necesarios para que el servidor web pueda trabajar con ellos.

Imagen de la base de datos

La imagen de la base de datos utiliza MariaDB 11.4 como base e incorpora los scripts SQL necesarios para inicializar la estructura de S.I.G.S.M.

Estos scripts se copian al directorio /docker-entrypoint-initdb.d/, utilizado por la imagen oficial de MariaDB para ejecutar automáticamente los archivos de inicialización cuando se crea una base de datos nueva.

También se incorpora el archivo seedDemo.sql, que contiene información ficticia destinada a los entornos de prueba y demostración.

Adicionalmente, durante la construcción de las imágenes se almacena el identificador del commit de GitHub utilizado. Esto permite consultar posteriormente qué versión del código fue incorporada a cada imagen.

20.4.3. Configuración del archivo compose.yaml

El archivo compose.yaml constituye el elemento principal de la configuración de Docker Compose, ya que permite definir en un único lugar los servicios necesarios para ejecutar S.I.G.S.M.

En primer lugar, se establece sigsm como nombre del proyecto y se definen los cinco servicios descritos anteriormente: frontend, backend, db, admin-init y seed-demo.

Para cada servicio se indican los parámetros correspondientes a su construcción y ejecución, incluyendo las imágenes utilizadas, las variables de entorno, los directorios persistentes y la red a la que debe conectarse.

Una de las configuraciones más relevantes corresponde a la publicación de puertos. Únicamente el servicio frontend publica un puerto en el equipo anfitrión, utilizando por defecto el 8080 para permitir el acceso a la aplicación desde un navegador.

Por su parte, el backend y la base de datos se comunican mediante la red interna de Docker, sin necesidad de publicar directamente sus puertos hacia el exterior.

También se definen dos volúmenes persistentes, denominados db_data y docs_data, destinados a conservar respectivamente la información de MariaDB y los archivos almacenados por el módulo de documentación.

Otra configuración importante corresponde a las dependencias entre servicios, establecidas mediante depends_on. En particular, se utiliza una comprobación de estado para asegurar que MariaDB se encuentre disponible antes de iniciar los procesos que requieren acceder a la base de datos.

Los servicios principales también utilizan la política de reinicio unless-stopped, permitiendo que Docker intente reiniciarlos automáticamente si se detienen de forma inesperada, salvo que hayan sido detenidos explícitamente.

Por último, se utiliza la funcionalidad de perfiles de Docker Compose para controlar la ejecución del servicio seed-demo. Este servicio pertenece al perfil demo, por lo que solamente se incorpora cuando se solicita expresamente una instalación con datos de demostración.

20.4.4. Configuración de Apache y Nginx

Para permitir el funcionamiento conjunto del frontend y backend se prepararon archivos de configuración específicos para Nginx y Apache HTTP Server.

Estas configuraciones establecen cómo deben atenderse las solicitudes HTTP y cómo se realiza la comunicación entre ambos servicios.

Configuración de Nginx

El archivo nginx.conf define el comportamiento del servidor web encargado de proporcionar la interfaz de S.I.G.S.M.

Nginx utiliza el directorio /usr/share/nginx/html para servir los archivos generados durante la compilación del frontend. También se establece index.html como archivo principal de la aplicación.

Debido a que React utiliza rutas internas para mostrar las diferentes pantallas, se implementó una configuración mediante try_files que permite devolver index.html cuando una ruta solicitada no corresponde a un archivo físico existente.

Esto evita errores al actualizar el navegador o acceder directamente a una dirección interna de la aplicación.

Adicionalmente, se configuró Nginx como proxy inverso. Las solicitudes dirigidas a rutas que comienzan con /api/ son enviadas al contenedor backend mediante la dirección http://backend.

También se estableció un límite de 25 MB para el tamaño del cuerpo de las solicitudes HTTP, teniendo en cuenta que el sistema permite cargar documentos mediante formularios.

Configuración de Apache

El archivo apache.conf define el funcionamiento del servidor web utilizado por la API desarrollada en PHP.

Se configuró el directorio /var/www/api/public como raíz pública del servidor y se estableció un alias para atender las solicitudes realizadas bajo el prefijo /api.

Además, se incorporaron reglas de reescritura de URL mediante mod_rewrite, permitiendo que determinadas rutas sean procesadas por los archivos index.php correspondientes sin necesidad de incluir explícitamente este nombre en la dirección solicitada.

También se deshabilitó el listado de directorios mediante la opción -Indexes, evitando que Apache muestre automáticamente el contenido de una carpeta cuando no existe un archivo índice.

Los registros de acceso y errores se redirigen hacia las salidas estándar del contenedor, permitiendo consultarlos mediante las herramientas de administración de Docker.

Configuración adicional de PHP

Junto con Apache se incorpora un archivo php.ini que permite establecer determinados parámetros necesarios para el funcionamiento del backend.

Entre las configuraciones aplicadas se encuentran la zona horaria America/Montevideo, un tamaño máximo de archivo de 20 MB, un límite de 24 MB para las solicitudes POST, un límite de memoria de 256 MB y un tiempo máximo de ejecución de 60 segundos.

También se deshabilita la visualización de errores de PHP en las respuestas y se mantiene habilitado su registro, evitando mostrar directamente información técnica del servidor a los usuarios.

20.4.5. Variables de entorno y archivo .env

Para evitar incorporar credenciales directamente en los archivos de configuración de Docker Compose, se implementó un mecanismo basado en variables de entorno.

Estas variables se definen en el archivo .env, ubicado dentro del directorio de Docker, y permiten establecer los parámetros necesarios para ejecutar los diferentes servicios.

El repositorio incluye un archivo denominado .env.example, que funciona como plantilla para preparar la configuración inicial sin publicar las contraseñas reales utilizadas durante el despliegue.

Las principales variables definidas son las siguientes:

Variable	Función
WEB_PORT	Puerto utilizado para acceder a la aplicación web.
DB_USER	Usuario de MariaDB utilizado por el backend.
DB_PASSWORD	Contraseña del usuario de la base de datos.
MARIADB_ROOT_PASSWORD	Contraseña administrativa de MariaDB.
ADMIN_NAME	Nombre del administrador inicial del sistema.
ADMIN_EMAIL	Correo electrónico utilizado para iniciar sesión.
ADMIN_PASSWORD	Contraseña inicial del administrador.

Durante el proceso de instalación se debe crear el archivo .env a partir de la plantilla proporcionada y sustituir las credenciales de ejemplo por valores seguros.

Docker Compose utiliza estas variables para configurar los servicios. En particular, las credenciales de MariaDB se proporcionan al contenedor de base de datos y al backend, mientras que la información del administrador inicial es utilizada por el servicio admin-init.

Es importante diferenciar las credenciales de la base de datos de las correspondientes a los funcionarios de S.I.G.S.M. Las primeras permiten establecer la comunicación entre los servicios, mientras que las segundas se utilizan para acceder a las funcionalidades de la aplicación.

Como medida de seguridad, el archivo .env se encuentra excluido del control de versiones mediante .gitignore, evitando su incorporación accidental al repositorio público.

De esta manera, se mantiene separada la configuración del entorno respecto del código fuente, facilitando la instalación del sistema en distintos servidores sin necesidad de modificar directamente los archivos de la aplicación.

20.5. Despliegue de la aplicación

Una vez definida la arquitectura de contenedores y preparados los archivos de configuración, se procedió a implementar el entorno de despliegue de S.I.G.S.M. sobre el servidor Fedora Server configurado previamente en VirtualBox.

El objetivo de este procedimiento es permitir que la aplicación pueda instalarse y ejecutarse a partir de los archivos disponibles en GitHub, sin necesidad de configurar manualmente los servidores web, las dependencias de PHP o el gestor de base de datos.

Para ello, se utiliza Docker Compose como herramienta principal, encargada de construir las imágenes, crear los recursos necesarios e iniciar los servicios en el orden establecido.

20.5.1. Preparación del servidor

Antes de iniciar el despliegue, es necesario disponer de un servidor con Docker Engine, Docker Compose y Git correctamente instalados.

En nuestro caso se utilizó Fedora Server 43, ejecutado dentro de una máquina virtual de VirtualBox. La instalación y configuración inicial del sistema operativo, así como la instalación de Docker Engine, fueron documentadas en apartados anteriores.

Como comprobación previa, se utilizaron los siguientes comandos:

docker --version
docker compose version
git --version

Estos comandos permiten verificar que las herramientas necesarias se encuentran disponibles.

También se comprobó la comunicación con el motor de Docker mediante: **docker info**

Adicionalmente, se configuraron los permisos del usuario admin para permitirle administrar los contenedores sin tener que ejecutar todos los comandos mediante sudo.

Para ello, se incorporó el usuario al grupo docker, permitiendo ejecutar las herramientas de administración directamente desde su sesión.

Esta configuración también resulta relevante para las tareas automatizadas de respaldo, ya que estas se ejecutan mediante el servicio cron utilizando los permisos del usuario que las programa.

Finalmente, se verificó que el servidor dispusiera de conectividad a Internet para descargar las imágenes base y obtener los archivos necesarios desde GitHub.

20.5.2. Obtención de los archivos desde GitHub

Los archivos necesarios para desplegar S.I.G.S.M. se encuentran publicados en el repositorio del proyecto: https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU

Debido a que el servidor está destinado principalmente a ejecutar la aplicación, se preparó un procedimiento que permite obtener únicamente el directorio correspondiente a Docker, evitando descargar localmente la totalidad del código fuente y la documentación.

Para ello, puede utilizarse la funcionalidad sparse-checkout de Git.

En primer lugar, se clona el repositorio mediante:
git clone --depth 1 --filter=blob:none --sparse --branch main https://github.com/MrGalletah/S.I.G.S.M---Proyecto-UTU.git

Posteriormente, se accede al repositorio y se selecciona el directorio de Docker:

cd S.I.G.S.M---Proyecto-UTU
git sparse-checkout set prototipos/docker

Finalmente, se accede a la carpeta que contiene el archivo compose.yaml:

cd prototipos/docker

Este procedimiento permite disponer de los archivos necesarios para construir y ejecutar los contenedores.

No es imprescindible descargar localmente el código completo del frontend, backend o base de datos, dado que sus respectivos Dockerfiles obtienen los archivos correspondientes directamente desde GitHub durante la construcción de las imágenes.

De esta manera, se simplifica la preparación del servidor y se mantiene una separación entre los archivos utilizados para administrar el despliegue y el código de los diferentes componentes de la aplicación.

20.5.3. Configuración inicial del entorno

Antes de iniciar los contenedores, es necesario configurar las variables de entorno utilizadas por Docker Compose.

Para ello, el proyecto proporciona un archivo .env.example, que contiene una plantilla con los parámetros necesarios para ejecutar los servicios.

Desde el directorio de Docker se crea una copia de dicho archivo mediante:
cp .env.example .env

Posteriormente, se edita el archivo .env para establecer las credenciales y los parámetros correspondientes al entorno.

Entre los valores configurados se encuentran el puerto de acceso al frontend, el usuario y contraseña de MariaDB, la contraseña administrativa de la base de datos y las credenciales del administrador inicial de S.I.G.S.M.

Es importante sustituir los valores de ejemplo por contraseñas seguras antes de realizar la instalación. En particular, la contraseña del administrador inicial debe tener una longitud mínima de 12 caracteres, requisito establecido por el script encargado de crear dicha cuenta.

Una vez completada la configuración, puede comprobarse la validez del archivo de Docker Compose mediante el siguiente comando: **docker compose config --quiet**

Esta instrucción permite detectar errores en la configuración y comprobar que se hayan definido las variables obligatorias, sin imprimir las credenciales utilizadas.

El archivo .env debe conservarse en el servidor y no incorporarse al repositorio público, debido a que contiene información sensible necesaria para establecer las conexiones entre los diferentes servicios.

20.5.4. Construcción e inicio de los contenedores

Una vez preparada la configuración, se procede a construir las imágenes e iniciar los servicios mediante Docker Compose.

Para realizar una instalación normal de S.I.G.S.M. se utiliza el siguiente comando:
docker compose up -d --build

Este comando permite construir las imágenes definidas en los Dockerfiles y crear los contenedores necesarios para ejecutar la aplicación.

Durante el proceso, Docker obtiene las imágenes base de las tecnologías utilizadas, descarga el código correspondiente desde GitHub e incorpora las dependencias y configuraciones necesarias.

Además, Docker Compose crea la red interna del proyecto y los volúmenes persistentes utilizados para almacenar la base de datos y los documentos.

La opción --build solicita la construcción de las imágenes, mientras que -d permite ejecutar los servicios en segundo plano, liberando la terminal una vez iniciado el despliegue.

La configuración también establece dependencias entre los servicios. En particular, MariaDB debe completar correctamente su inicialización antes de que pueda ejecutarse el proceso de creación del administrador. Posteriormente, el backend puede iniciar su funcionamiento y atender las solicitudes provenientes del frontend.

Gracias a esta configuración, es posible poner en funcionamiento los componentes principales de S.I.G.S.M. mediante un único comando, sin necesidad de iniciar manualmente Apache, Nginx o MariaDB.

20.5.5. Inicialización de la base de datos

Durante la primera instalación, el contenedor de MariaDB debe crear la estructura de base de datos necesaria para el funcionamiento de S.I.G.S.M.

Para automatizar este procedimiento se incorporaron los scripts SQL del proyecto a la imagen de MariaDB, utilizando el directorio /docker-entrypoint-initdb.d/.

Los archivos de inicialización son los siguientes:

- dbSchema_01.sql
- dbSchema_02.sql
- dbSchema_03.sql
- dbSchema_04.sql

Estos scripts se ejecutan automáticamente durante la creación inicial de la base de datos y permiten generar las tablas, relaciones, restricciones y catálogos correspondientes a los diferentes módulos del sistema.

La base de datos utiliza el nombre sigsm y se configura para trabajar con el conjunto de caracteres utf8mb4, permitiendo almacenar correctamente caracteres especiales y diferentes idiomas.

Una característica importante de este mecanismo es que los scripts de inicialización solamente se ejecutan cuando MariaDB detecta que su directorio de datos todavía no ha sido inicializado.

Por este motivo, reconstruir la imagen del contenedor no provoca que las tablas existentes sean eliminadas ni que los scripts SQL vuelvan a ejecutarse automáticamente.

Para modificar la estructura de una base de datos que ya contiene información, es necesario aplicar las modificaciones SQL correspondientes, conservando los datos existentes.

Adicionalmente, se configuró una comprobación de estado mediante healthcheck, que verifica la disponibilidad de MariaDB y la existencia de elementos de la estructura necesarios para continuar con la inicialización del sistema.

20.5.6. Creación del administrador inicial

Una vez inicializada la base de datos, Docker Compose ejecuta el servicio auxiliar admin-init, encargado de crear la primera cuenta administrativa de S.I.G.S.M.

Este procedimiento permite disponer de un usuario autorizado para acceder a la aplicación sin necesidad de insertar manualmente sus datos desde una terminal SQL o mediante una herramienta de administración de bases de datos.

El servicio utiliza un script PHP denominado bootstrap-admin.php, que obtiene las credenciales establecidas previamente en el archivo .env.

Durante su ejecución, el script comprueba que exista el rol de Administrador en la base de datos y verifica si ya se encuentra registrado un funcionario con el correo electrónico proporcionado.

Si el usuario todavía no existe, se crea el registro correspondiente, se genera un hash seguro de su contraseña y se establece la relación con el rol administrativo.

En caso de que ya exista un administrador con el mismo correo y rol, el procedimiento finaliza sin modificar su contraseña.

Esta comprobación permite evitar la creación de cuentas duplicadas cuando el servicio vuelve a ejecutarse sobre una instalación existente.

Una vez completado el proceso, el contenedor admin-init finaliza su ejecución, ya que no necesita permanecer activo durante el funcionamiento habitual de la aplicación.

El resultado puede comprobarse mediante: **docker compose logs admin-init**

Este comando permite consultar los mensajes generados durante la creación del administrador e identificar posibles errores de configuración o conexión con MariaDB.

20.5.7. Carga opcional de datos de demostración

Además de la instalación normal, se preparó una modalidad que permite ejecutar S.I.G.S.M. con información ficticia destinada a pruebas y demostraciones.

Para ello, se incorporó el servicio auxiliar seed-demo, configurado mediante un perfil opcional de Docker Compose.

En una instalación nueva, este perfil puede activarse utilizando el siguiente comando: **docker compose --profile demo up -d --build**

Este procedimiento realiza el despliegue de los servicios principales y ejecuta adicionalmente el script seedDemo.sql, encargado de insertar datos de ejemplo en la base de datos.

La información generada incluye funcionarios, asignaciones de roles, vehículos, categorías, traslados, encuestas, preguntas y respuestas, entre otros registros.

En particular, se generan traslados con diferentes estados y situaciones, permitiendo comprobar funcionalidades como la solicitud, asignación de recursos, seguimiento e historial.

El servicio seed-demo se ejecuta después de que MariaDB se encuentre disponible y el administrador inicial haya sido creado correctamente.

Una vez completada la carga, finaliza automáticamente su ejecución.

El resultado puede consultarse mediante: **docker compose --profile demo logs seed-demo**

Es importante destacar que este procedimiento está destinado a instalaciones nuevas y entornos de prueba. El script comprueba determinadas condiciones de la base de datos antes de insertar los registros, evitando que se mezclen los datos ficticios con información ya existente.

Por lo tanto, no debe ejecutarse repetidamente sobre una instalación en uso.

Para iniciar normalmente una aplicación que ya dispone de información almacenada, se utiliza: **docker compose up -d** . Sin activar el perfil de demostración.

20.5.8. Verificación del despliegue

Finalizada la instalación, es necesario comprobar que los servicios se hayan iniciado correctamente y que la aplicación pueda comunicarse con la base de datos.

La primera comprobación consiste en consultar el estado de los contenedores mediante: **docker compose ps**

Este comando permite visualizar los servicios activos, sus estados y los puertos publicados.

También puede utilizarse: **docker compose ps -a**

La opción -a permite incluir los contenedores que finalizaron su ejecución, como admin-init y, cuando se utiliza el perfil correspondiente, seed-demo.

Es normal que estos servicios auxiliares aparezcan con estado Exited (0), ya que fueron diseñados para completar una tarea específica y finalizar correctamente.

Por su parte, los servicios frontend, backend y db deben permanecer en ejecución. MariaDB también debe indicar que su comprobación de estado se ha completado satisfactoriamente.

Para revisar los mensajes generados por los contenedores puede utilizarse: **docker compose logs**

También es posible consultar individualmente los registros de cada servicio, por ejemplo:

docker compose logs backend
docker compose logs db
docker compose logs frontend

Una vez verificado el estado de los servicios, se procede a acceder a la aplicación desde un navegador web.

Si el acceso se realiza directamente desde el equipo que ejecuta Docker y se utiliza el puerto predeterminado, la dirección es: **http://localhost:8080**

Cuando el acceso se realiza desde otro equipo de la red, debe utilizarse la dirección IP del servidor Fedora junto con el puerto configurado, siempre que la configuración de red y el firewall permitan dicha conexión.

Finalmente, se comprueba el funcionamiento del inicio de sesión utilizando las credenciales correspondientes al administrador inicial.

Esta prueba permite verificar conjuntamente el funcionamiento del frontend, la comunicación con el backend y el acceso a la información almacenada en MariaDB.

Como comprobación adicional, las imágenes construidas incorporan el identificador del commit de GitHub utilizado durante su creación. Por ejemplo, es posible consultar el identificador correspondiente al backend mediante:
docker compose exec backend cat /etc/sigsm-commit

Este mecanismo permite identificar la versión del código utilizada en el despliegue y resulta útil para comprobar futuras actualizaciones.

Con estas verificaciones se completa el procedimiento de instalación, quedando S.I.G.S.M. disponible para su utilización dentro del entorno de pruebas configurado.

20.6. Persistencia y respaldos

La persistencia de datos constituye un aspecto fundamental del despliegue de S.I.G.S.M., dado que el sistema almacena información que debe conservarse independientemente del estado de los contenedores utilizados para ejecutar la aplicación.

Para garantizar esta conservación, se implementaron volúmenes persistentes de Docker destinados a almacenar la base de datos y los documentos cargados por los usuarios. Adicionalmente, se adaptó el sistema de respaldos desarrollado anteriormente para permitir la generación de copias de seguridad de estos datos, tanto de forma manual como automatizada.

Esta implementación permite mantener la información separada de los contenedores y disponer de respaldos que faciliten su recuperación ante posibles fallos, pérdidas de datos o problemas durante el mantenimiento del servidor.

20.6.1. Volúmenes utilizados por la aplicación

Por defecto, los archivos generados dentro de un contenedor forman parte de su sistema de archivos y pueden perderse cuando dicho contenedor es eliminado y sustituido por otro.

Para evitar este comportamiento, Docker permite utilizar volúmenes persistentes, que mantienen la información almacenada independientemente del ciclo de vida de los contenedores.

En S.I.G.S.M. se configuraron dos volúmenes mediante el archivo compose.yaml:

Volumen	Servicio	Directorio de montaje	Contenido
db_data	db	/var/lib/mysql	Archivos de la base de datos MariaDB
docs_data	backend	/var/www/api/storage	Documentos y archivos almacenados por la aplicación

El volumen db_data se utiliza para almacenar los archivos internos de MariaDB, mientras que docs_data conserva los documentos cargados mediante las funcionalidades del sistema.

Estos volúmenes son administrados por Docker y, debido a que el proyecto se identifica mediante el nombre sigsm, normalmente reciben los nombres sigsm_db_data y sigsm_docs_data.

Para consultar los volúmenes existentes en el servidor se utiliza: **docker volume ls**

La definición de los volúmenes se encuentra integrada en Docker Compose, por lo que se crean automáticamente cuando se ejecuta la aplicación por primera vez.

Esta configuración evita la necesidad de establecer manualmente directorios de almacenamiento dentro del sistema operativo anfitrión y permite mantener separados los datos de las imágenes que contienen el código y las dependencias de la aplicación.

20.6.2. Persistencia de MariaDB y documentos

La utilización de volúmenes persistentes permite realizar determinadas operaciones de administración sin perder la información almacenada.

En el caso de MariaDB, el volumen conserva las tablas, registros, relaciones y demás elementos de la base de datos sigsm. Esto permite detener o reconstruir el contenedor sin que sea necesario inicializar nuevamente toda su estructura.

De manera similar, el volumen de documentos mantiene los archivos físicos almacenados por el backend. Esta característica resulta especialmente importante para el módulo de documentación, ya que la información registrada en MariaDB debe corresponderse con los archivos disponibles en el almacenamiento.

Para recuperar completamente el funcionamiento de este módulo no basta con disponer únicamente de la base de datos, dado que esta almacena referencias y metadatos de los documentos, mientras que los archivos se conservan de forma independiente.

Por este motivo, se consideró necesario mantener ambos mecanismos de almacenamiento y respaldarlos conjuntamente.

Es importante destacar que la persistencia de los volúmenes no sustituye a una política de respaldos. Aunque permiten conservar los datos durante la recreación de contenedores, no protegen por sí solos frente a la eliminación accidental de información, fallos de almacenamiento o pérdida de la máquina virtual.

Asimismo, determinadas operaciones de Docker pueden eliminar los volúmenes de manera explícita. Por ejemplo, el comando docker compose down -v elimina los contenedores y los volúmenes declarados por el proyecto, provocando la pérdida de los datos almacenados en ellos.

Por estas razones, se mantuvo la política de respaldos definida anteriormente y se amplió para incorporar la información gestionada mediante Docker.

20.6.3. Adaptación de los scripts de respaldos a Docker

Durante las primeras etapas del proyecto se desarrolló un sistema de respaldos mediante Bash, orientado a la instalación de XAMPP utilizada para ejecutar Apache y MariaDB.

Este sistema permitía generar respaldos locales, transferir archivos a otro servidor mediante SSH y realizar copias de seguridad de la base de datos. También incorporaba funcionalidades para programar tareas automáticas utilizando cron.

Con la incorporación de Docker surgió la necesidad de adaptar este mecanismo, dado que MariaDB y los documentos dejaron de depender directamente de los directorios utilizados por XAMPP y pasaron a almacenarse en volúmenes administrados por Docker.

Para mantener la compatibilidad con el entorno anterior, se decidió conservar las funciones existentes e incorporar una nueva función denominada backupDocker() dentro del archivo backups.sh.

Esta función permite generar un respaldo completo de los datos persistentes de la aplicación mediante el siguiente procedimiento:

Respaldo de la base de datos

En primer lugar, el script accede al contenedor de MariaDB mediante docker compose exec y ejecuta la herramienta mariadb-dump, encargada de exportar la información de la base de datos.

Para ello se utiliza el siguiente comando:
docker compose exec -T db sh -c 'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" mariadb-dump -u root --single-transaction sigsm'

La opción -T evita la asignación de una terminal interactiva, facilitando la ejecución del comando desde tareas automatizadas.

Por su parte, --single-transaction permite generar un volcado consistente de las tablas transaccionales sin bloquear su lectura durante todo el proceso.

El resultado se redirige a un archivo SQL temporal en el servidor anfitrión, que posteriormente será incorporado al respaldo comprimido.

La contraseña de MariaDB se obtiene de las variables de entorno disponibles dentro del contenedor, evitando incorporarla directamente al código del script.

Respaldo de documentos

Una vez exportada la base de datos, el script utiliza docker compose cp para copiar los archivos almacenados en el directorio persistente del backend.

El comando utilizado es: **docker compose cp backend:/var/www/api/storage/. "$TEMP/documentos/"**

De esta manera, los archivos se extraen desde el almacenamiento utilizado por el contenedor y se copian a un directorio temporal del sistema anfitrión.

Esto permite respaldar los documentos sin necesidad de acceder directamente a las rutas internas en las que Docker administra sus volúmenes.

Compresión y almacenamiento

Después de obtener la base de datos y los documentos, ambos elementos se incorporan a un único archivo comprimido en formato .tar.gz.

Los respaldos se almacenan en el directorio /home/admin/Backups, utilizando un nombre que incluye la fecha y hora de creación.

El archivo resultante sigue el formato: **sigsmDocker_DD-MM-AAAA_HH-MM.tar.gz**

Dentro del respaldo se incluyen el archivo sigsm.sql, con la información de MariaDB, y el directorio documentos, con los archivos persistentes del backend.

Una vez finalizada correctamente la compresión, se eliminan los archivos temporales utilizados durante el procedimiento.

También se incorporaron comprobaciones del resultado de las operaciones principales. Si falla la exportación de MariaDB, la copia de documentos o la compresión, la función informa del error y evita considerar el respaldo como completado.

Adicionalmente, se utiliza umask 077 para restringir los permisos de los archivos generados, reduciendo la posibilidad de que otros usuarios del servidor accedan a la información respaldada.

La función se incorporó como una opción adicional del menú principal de backups.sh, permitiendo generar respaldos de Docker manualmente sin modificar las funcionalidades desarrolladas previamente para XAMPP.

Este procedimiento facilita la conservación conjunta de los datos principales de S.I.G.S.M. en un único archivo, que posteriormente puede almacenarse localmente o transferirse a otro servidor.

20.6.4. Automatización de respaldos mediante cron

Además de permitir la generación manual de respaldos, se adaptó la función createCron() del script principal para incorporar los servicios ejecutados mediante Docker.

Anteriormente, esta función permitía establecer una programación y seleccionar si el respaldo se realizaría de forma local o remota, utilizando scripts independientes destinados al entorno XAMPP.

Con la modificación realizada, el usuario puede seleccionar también el origen de los datos que desea respaldar, diferenciando entre XAMPP y Docker.

El procedimiento solicita los cinco parámetros de programación de cron: minuto, hora, día del mes, mes y día de la semana. Posteriormente, permite seleccionar el entorno de origen y el destino del respaldo.

Cuando se selecciona XAMPP, se mantienen los scripts cron/local.sh y cron/remote.sh utilizados anteriormente.

En cambio, cuando se selecciona Docker, se utiliza directamente la función backupDocker() del script principal, evitando crear archivos adicionales que duplicarían la lógica de generación de respaldos.

Para ello, cron ejecuta Bash, carga las funciones de backups.sh mediante source e invoca la función correspondiente.

Como ejemplo, una programación local diaria a las 04:00 podría generar la siguiente entrada: **0 4 * * * bash -c 'source /home/admin/scripts/backups.sh; backupDocker'**

Esta instrucción permite ejecutar el respaldo automáticamente, sin necesidad de abrir el menú interactivo del script.

Para conseguirlo, se modificó el archivo principal de manera que dicho menú solamente se muestre cuando el script se ejecuta directamente. Cuando se carga mediante source, únicamente se incorporan sus funciones al entorno de Bash.

Respaldos remotos

En el caso de los respaldos remotos de Docker, se utiliza el mismo procedimiento de generación local y posteriormente se transfiere el archivo comprimido al servidor de respaldos mediante scp.

En la configuración utilizada, el servidor remoto se identifica mediante la dirección 192.168.56.102 y el directorio de destino /home/admin/Backups.

La transferencia se ejecuta solamente cuando la función backupDocker() finaliza correctamente, evitando enviar archivos cuando se detecta un error en la generación del respaldo.

De esta manera, el archivo comprimido permanece disponible en el servidor local y se conserva una segunda copia en el servidor remoto.

Para permitir la ejecución automática de la transferencia, es necesario que la conexión SSH entre ambos servidores se encuentre configurada para funcionar sin solicitar una contraseña de forma interactiva.

Permisos de ejecución y administración de cron

Durante las pruebas se detectó que la ejecución del script mediante sudo provocaba que las tareas se crearan en el crontab de root, en lugar del correspondiente al usuario admin.

Esto se debía a que cron administra las tareas de manera independiente para cada usuario del sistema.

Para solucionar este inconveniente, se incorporó el usuario admin al grupo docker, permitiéndole ejecutar los comandos de Docker sin utilizar sudo.

De esta forma, el usuario puede ejecutar el script de respaldos y administrar sus propias tareas programadas, conservando las configuraciones de cron que ya se habían establecido anteriormente.

Esta decisión también permite que los respaldos automatizados se ejecuten con los permisos necesarios para acceder a los contenedores.

Debe tenerse en cuenta que la pertenencia al grupo docker concede privilegios elevados sobre el servidor, motivo por el cual su utilización debe limitarse a usuarios administrativos autorizados.

Comprobación de las tareas programadas

Una vez creada una programación, puede consultarse mediante: **crontab -l**

Este comando permite visualizar las tareas registradas para el usuario actual y comprobar que las nuevas entradas se hayan incorporado correctamente.

La frecuencia de ejecución puede configurarse según las necesidades del entorno, manteniendo como referencia la política de respaldos locales diarios y remotos semanales definida anteriormente.

Finalmente, es importante señalar que la generación de un archivo comprimido no garantiza por sí sola la posibilidad de recuperar el sistema. Por ello, los respaldos deben conservarse adecuadamente y someterse a pruebas periódicas de restauración.

Con estas modificaciones se amplió el sistema de respaldos existente para contemplar tanto el entorno tradicional basado en XAMPP como la nueva arquitectura de contenedores, manteniendo una administración centralizada desde el mismo script Bash.

20.7. Administración y mantenimiento de los contenedores

Una vez desplegada la aplicación, es necesario disponer de mecanismos que permitan administrar los servicios, supervisar su funcionamiento y aplicar actualizaciones cuando se realizan modificaciones en el código fuente.

Docker Compose proporciona diferentes comandos para realizar estas tareas desde la terminal del servidor, permitiendo gestionar los contenedores de S.I.G.S.M. de manera centralizada.

Las operaciones descritas en este apartado se realizan desde el directorio prototipos/docker, donde se encuentra el archivo compose.yaml utilizado para definir los servicios.

20.7.1. Inicio, detención y reinicio de servicios

Docker Compose permite controlar el estado de los contenedores mediante diferentes comandos, sin necesidad de administrar individualmente los procesos de Apache, Nginx o MariaDB.

Para iniciar los servicios de la aplicación se utiliza:

docker compose up -d

Este comando crea e inicia los contenedores necesarios según la configuración definida, utilizando las imágenes disponibles. La opción -d permite que los servicios permanezcan ejecutándose en segundo plano.

Cuando únicamente se desea detener temporalmente los contenedores, sin eliminarlos, se utiliza:

docker compose stop

Esta operación detiene los servicios en ejecución, conservando sus contenedores y los datos almacenados en los volúmenes persistentes.

Posteriormente, es posible volver a iniciarlos mediante:

docker compose start

También se dispone de la posibilidad de reiniciar los servicios mediante:

docker compose restart

Este procedimiento resulta útil cuando es necesario reiniciar los procesos de los contenedores existentes sin reconstruir sus imágenes.

Adicionalmente, Docker Compose permite actuar sobre un servicio específico. Por ejemplo, para reiniciar únicamente el backend se utiliza:

docker compose restart backend

De esta manera, es posible realizar determinadas operaciones de mantenimiento sin detener deliberadamente todos los componentes del sistema.

Otra operación disponible consiste en eliminar los contenedores y la red creada por Docker Compose mediante:

docker compose down

A diferencia de stop, este comando elimina los contenedores del proyecto, aunque conserva los volúmenes persistentes cuando no se indica expresamente su eliminación.

Esta característica permite recrear los servicios posteriormente sin perder la información almacenada en MariaDB o los documentos de la aplicación.

20.7.2. Consulta de estados y registros

Para comprobar el funcionamiento de S.I.G.S.M. se utilizan las herramientas de supervisión proporcionadas por Docker.

Una de las operaciones más habituales consiste en consultar los servicios administrados por Docker Compose mediante:

docker compose ps

Este comando permite visualizar los contenedores activos, sus estados y los puertos publicados en el equipo anfitrión.

Para incluir también los contenedores que finalizaron su ejecución se utiliza:

docker compose ps -a

Esta opción resulta especialmente útil para comprobar el resultado de los servicios auxiliares admin-init y seed-demo, que están diseñados para completar sus tareas y finalizar.

Además de consultar el estado de los contenedores, Docker permite visualizar los registros generados durante su ejecución.

Para consultar los registros generales se utiliza:

docker compose logs

Cuando es necesario analizar un servicio específico, puede indicarse su nombre. Por ejemplo:

docker compose logs backend

De esta manera, es posible identificar errores relacionados con la ejecución de PHP, solicitudes HTTP o problemas detectados por Apache.

También es posible consultar los registros de MariaDB o Nginx mediante:

docker compose logs db
docker compose logs frontend

Para observar los mensajes en tiempo real se utiliza la opción -f, como se muestra a continuación:

docker compose logs -f backend

Este comando mantiene abierta la visualización de los registros y permite observar los nuevos mensajes generados por el servicio.

La consulta de registros resulta especialmente importante durante las pruebas de la aplicación, ya que permite detectar problemas de comunicación entre contenedores, errores de conexión con la base de datos o fallos durante la ejecución de los endpoints.

Por otra parte, Docker también proporciona herramientas para supervisar el consumo de recursos del servidor.

Mediante el comando:

docker stats

Es posible consultar información sobre el uso de CPU, memoria y otros recursos consumidos por los contenedores activos.

Estas herramientas facilitan las tareas de mantenimiento y permiten identificar problemas sin necesidad de acceder directamente a la configuración interna de cada servicio.

20.7.3. Actualización de la aplicación desde GitHub

Una de las características de la implementación de Docker en S.I.G.S.M. es que las imágenes obtienen el código fuente directamente desde la rama main del repositorio de GitHub durante su construcción.

Esto permite incorporar nuevas versiones del frontend o backend sin necesidad de copiar manualmente los archivos de la aplicación al servidor.

Sin embargo, es importante destacar que los contenedores no se actualizan automáticamente cuando se realizan cambios en GitHub. Para incorporar dichas modificaciones es necesario reconstruir las imágenes correspondientes y recrear los contenedores que las utilizan.

Actualización del frontend

Cuando se realizan modificaciones en la interfaz desarrollada con React, como cambios en componentes, estilos o funcionalidades, es posible reconstruir únicamente la imagen del frontend.

Para ello, se utiliza:

docker compose build --no-cache frontend

La opción --no-cache permite realizar una construcción sin reutilizar las capas almacenadas previamente, asegurando que se vuelva a obtener el código disponible en GitHub.

Una vez construida la nueva imagen, se recrea el contenedor mediante:

docker compose up -d --no-deps frontend

La opción --no-deps permite aplicar la actualización sin iniciar o recrear deliberadamente las dependencias del servicio.

De esta manera, se puede actualizar la interfaz sin necesidad de reconstruir la imagen de MariaDB o modificar directamente el backend.

Actualización del backend

El procedimiento es similar cuando se realizan modificaciones en los endpoints, servicios o demás componentes de la API desarrollada en PHP.

En primer lugar, se reconstruye la imagen:

docker compose build --no-cache backend

Posteriormente, se recrea el contenedor:

docker compose up -d --no-deps backend

Como los documentos se encuentran almacenados en un volumen persistente, estos se mantienen durante la recreación del contenedor, siempre que dicho volumen no sea eliminado.

Actualización conjunta

Cuando se modifican tanto el frontend como el backend, es posible reconstruir ambas imágenes mediante:

docker compose build --no-cache frontend backend

Posteriormente, se aplican las imágenes reconstruidas mediante:

docker compose up -d --no-deps frontend backend

Este procedimiento permite incorporar las modificaciones de ambos componentes sin reconstruir innecesariamente la imagen de MariaDB.

Por otra parte, cuando los cambios afectan a los archivos de configuración de Docker, como compose.yaml, los Dockerfiles o los archivos de Apache y Nginx, es necesario actualizar también los archivos disponibles en el directorio de despliegue.

Para ello se utiliza:

git pull origin main

Una vez obtenidas las modificaciones, se reconstruyen las imágenes correspondientes y se aplican los cambios mediante Docker Compose.

Es importante diferenciar estas situaciones: las modificaciones realizadas en el código de la aplicación requieren reconstruir sus imágenes, mientras que los cambios realizados en los archivos de configuración del despliegue requieren actualizar también dichos archivos en el servidor.

Finalmente, cada imagen incorpora el identificador del commit de GitHub utilizado durante su construcción.

Por ejemplo, para consultar la versión del frontend puede utilizarse:

docker compose exec frontend cat /etc/sigsm-commit

Este mecanismo facilita la identificación de la versión desplegada y permite comprobar si los cambios publicados en el repositorio fueron incorporados correctamente.

20.8. Seguridad del entorno Docker

La seguridad constituye un aspecto importante en el despliegue de S.I.G.S.M., especialmente considerando que la aplicación está orientada a la gestión de información dentro de un entorno hospitalario.

Durante la implementación de Docker se aplicaron diferentes medidas destinadas a limitar la exposición de los servicios, administrar las credenciales y controlar los permisos de acceso al servidor.

No obstante, la configuración actual está orientada a un entorno académico y de pruebas, por lo que resulta necesario diferenciar las medidas implementadas de aquellas que serían necesarias antes de utilizar el sistema en producción.

20.8.1. Aislamiento de servicios y exposición de puertos

Una de las principales medidas adoptadas consiste en separar los componentes de la aplicación mediante contenedores independientes, evitando ejecutar todos los servicios directamente sobre el sistema operativo anfitrión.

Para permitir su comunicación se configuró una red virtual de Docker denominada internal, utilizando el controlador bridge. Esta red permite que los servicios se comuniquen mediante sus nombres sin necesidad de publicar todos sus puertos.

En la configuración actual, únicamente el contenedor frontend publica un puerto en el servidor, utilizando por defecto el puerto 8080.

El backend y MariaDB no publican sus puertos hacia el equipo anfitrión. De esta manera, las solicitudes HTTP dirigidas a la API deben pasar por Nginx, mientras que las conexiones a la base de datos se realizan desde los servicios autorizados por la arquitectura.

Adicionalmente, Apache tiene deshabilitado el listado de directorios y PHP está configurado para no mostrar directamente los errores de ejecución a los usuarios.

Estas medidas permiten reducir la exposición de componentes internos, aunque el aislamiento proporcionado por Docker no sustituye a los mecanismos de autenticación, autorización y seguridad propios de la aplicación.

20.8.2. Gestión de credenciales

Para administrar las credenciales utilizadas por los contenedores se implementó un sistema basado en variables de entorno, definidas mediante el archivo .env.

Este archivo contiene las contraseñas de MariaDB, las credenciales utilizadas por el backend para acceder a la base de datos y la información necesaria para crear el administrador inicial.

Como medida preventiva, se incluyó el archivo .env en .gitignore, evitando su incorporación accidental al repositorio público de GitHub.

En su lugar, se proporciona el archivo .env.example, que contiene únicamente valores de referencia para preparar una nueva instalación.

Durante la configuración del servidor se deben sustituir las contraseñas de ejemplo por credenciales seguras y diferentes para cada servicio.

Asimismo, el administrador inicial se crea mediante un script PHP que utiliza password_hash() para almacenar su contraseña de forma segura en la base de datos, evitando conservarla en texto plano dentro de los registros de funcionarios.

Sin embargo, es importante considerar que el archivo .env y las variables de entorno no constituyen un sistema especializado de gestión de secretos. Las credenciales pueden ser accesibles para usuarios con privilegios suficientes sobre Docker o el servidor.

Por este motivo, se debe restringir el acceso al archivo de configuración y evitar compartir su contenido mediante capturas, documentación pública o archivos incorporados al repositorio.

20.8.3. Permisos de administración de Docker

Durante la adaptación del sistema de respaldos se identificó la necesidad de permitir que el usuario admin pudiera ejecutar comandos de Docker sin utilizar sudo.

Inicialmente, la ejecución del script backups.sh con privilegios de administrador provocaba que las tareas programadas mediante cron se registraran en el crontab de root, en lugar de utilizar el correspondiente al usuario admin.

Para solucionar este inconveniente, se incorporó dicho usuario al grupo docker mediante el siguiente comando:

sudo usermod -aG docker admin

Posteriormente, se inició una nueva sesión y se comprobó que el usuario podía administrar los contenedores sin utilizar sudo.

Esta configuración permitió mantener las tareas programadas bajo el usuario admin, incluyendo los respaldos automatizados de MariaDB y los documentos almacenados en Docker.

No obstante, la pertenencia al grupo docker implica disponer de privilegios elevados sobre el equipo anfitrión, potencialmente equivalentes a los del usuario root.

Por este motivo, la asignación de estos permisos debe limitarse a usuarios administrativos de confianza. En un entorno de producción sería recomendable evaluar mecanismos más restrictivos para ejecutar las tareas automatizadas, evitando conceder acceso general a Docker cuando no resulte necesario.

20.8.4. Consideraciones para un despliegue en producción

Aunque la configuración implementada permite ejecutar S.I.G.S.M. de manera funcional dentro del entorno de pruebas, sería necesario incorporar medidas adicionales antes de utilizarla en una infraestructura hospitalaria real.

Una de las principales mejoras pendientes es la implementación de HTTPS mediante certificados TLS. Actualmente, el servidor Nginx se encuentra configurado para atender solicitudes HTTP, por lo que las comunicaciones no disponen de cifrado de transporte proporcionado por esta configuración.

También sería necesario revisar las reglas del firewall y restringir las conexiones entrantes a los puertos y redes estrictamente necesarios, evitando exponer la aplicación a equipos no autorizados.

En cuanto a las credenciales, se recomienda utilizar mecanismos de gestión de secretos más adecuados para producción, junto con políticas de renovación de contraseñas y control de acceso a los archivos de configuración.

Otra consideración importante corresponde al mantenimiento de las imágenes y dependencias utilizadas. Estas deberían actualizarse periódicamente para incorporar correcciones de seguridad, realizando previamente las pruebas necesarias para comprobar la compatibilidad de los cambios.

Asimismo, sería conveniente implementar mecanismos de supervisión, registro de eventos y detección de fallos que permitan identificar comportamientos anómalos en los servicios.

Finalmente, considerando la naturaleza de la información que podría gestionar S.I.G.S.M., sería imprescindible realizar una evaluación de seguridad más amplia, contemplando la protección de los datos personales, el control de acceso por roles, la auditoría de operaciones y el cumplimiento de las disposiciones legales aplicables.

Por lo tanto, el despliegue actual debe considerarse una implementación destinada al desarrollo, las pruebas y la demostración del sistema, que proporciona una base técnica sobre la cual podrían incorporarse las medidas adicionales necesarias para su utilización en producción.

20.9. Conclusiones

La implementación de Docker en S.I.G.S.M. permitió establecer un entorno de despliegue más organizado y reproducible, facilitando la instalación y administración de los diferentes componentes de la aplicación.

Mediante Docker Compose se consiguió integrar el frontend, backend y base de datos en una arquitectura basada en contenedores independientes, reduciendo la necesidad de configurar manualmente las tecnologías utilizadas en el servidor. Asimismo, la incorporación de los archivos de configuración al repositorio de GitHub facilita la distribución del sistema y su posterior actualización.

Otro aspecto importante fue la implementación de volúmenes persistentes para conservar la información de MariaDB y los documentos almacenados por la

aplicación. Esta configuración se complementó con la adaptación del sistema de respaldos desarrollado anteriormente, permitiendo generar copias de seguridad de los datos almacenados en Docker y automatizar su ejecución mediante cron.

Durante este proceso también se abordaron aspectos relacionados con la administración de contenedores, la configuración de redes internas, los permisos de usuario y la protección de las credenciales utilizadas por los servicios.

En conjunto, el trabajo realizado permitió avanzar desde un entorno de desarrollo basado en XAMPP hacia una solución de despliegue más estructurada y sencilla de mantener. Aunque todavía sería necesario incorporar medidas adicionales de seguridad para su utilización en producción, la implementación actual proporciona una base funcional para continuar desarrollando, probando y demostrando S.I.G.S.M.

21. Bibliografía

Amazon AWS. (n.d.). *Servidor privado virtual y alojamiento web - Amazon Lightsail*. AWS. Retrieved June 9, 2026, from https://aws.amazon.com/es/lightsail/

Apache Friends. (n.d.). *Download XAMPP*. Xampp. Retrieved June 9, 2026, from https://www.apachefriends.org/es/download.html

Boy, P., Fenzi, K., & Kuparinen, J. (2026, 4 28). *Fedora Server Installation Guide :: Fedora Docs*. Fedora Docs. Retrieved June 9, 2026, from https://docs.fedoraproject.org/en-US/fedora-server/installation/

Cotton, B. (n.d.). *Automatic Updates :: Fedora Docs*. Fedora Docs. Retrieved June 9, 2026, from https://docs.fedoraproject.org/en-US/quick-docs/autoupdates/

dnf5. (n.d.). *DFN5 Documentation*. DFN5 Documentation. Retrieved 6 9, 2026, from https://dnf5.readthedocs.io/en/stable/dnf5_plugins/automatic.8.html

Fedora Project. (n.d.). *Hardware Overview*. Hardware Overview. Retrieved 6 9, 2026, from https://docs.fedoraproject.org/en-US/fedora/latest/release-notes/hardware_overview/

Fedora Project. (n.d.). *Setting up a basic web server*. Setting up a basic web server. Retrieved 6 9, 2026, from https://docs.fedoraproject.org/en-US/fedora-server/services/httpd-basic-setup/

Fedora Project. (2026, 4 28). *Post Instalation Tasks*. Post Instalation Tasks. Retrieved 6 9, 2026, from Post Instalation Tasks

Fedora Server interactive local installation :: Fedora Docs. (2026). Fedora Docs. Retrieved May 15, 2026, from https://docs.fedoraproject.org/en-US/fedora-server/installation/interactive-local/

Git. (n.d.). *Reference*. Git. Retrieved June 9, 2026, from https://git-scm.com/docs

IBM. (n.d.). *¿Qué es un servidor bare metal?* IBM. Retrieved June 9, 2026, from https://www.ibm.com/es-es/think/topics/bare-metal-dedicated-servers

Instalar XAMPP 8.2.12 | Fedora. (2024, 5 1). Instalar XAMPP 8.2.12 | Fedora. Retrieved 6 9, 2026, from https://blog.linuxitos.com/post/instalar-xampp-8-2-12-fedora-40

MariaDB. (n.d.). *ACID: Concurrency Control with Transactions*. ACID: Concurrency Control with Transactions. Retrieved 6 9, 2026, from https://mariadb.com/docs/general-resources/database-theory/acid-concurrency-control-with-transactions

Mutai, J. (2026, 4 10). *Install XAMPP on Fedora*. Install XAMPP on Fedora. Retrieved 6 9, 2026, from https://computingforgeeks.com/how-to-install-xampp-on-fedora/

National Vulnerability Database (NIIST). (2025, January 15). *NVD - CVE-2024-52005*. NVD. Retrieved June 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2024-52005

National Vulnerability Database (NIST). (2023, 1 20). *CVE-2022-47015*. Retrieved 6 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2022-47015

National Vulnerability Database (NIST). (2023, December 24). *NVD - cve-2023-51767*. NVD. Retrieved June 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2023-51767

National Vulnerability Database (NIST). (2024, 10 17). *CVE-2023-26785*. Retrieved 6 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2023-26785

National Vulnerability Database (NIST). (2025, January 14). *NVD - cve-2024-50349*. NVD. Retrieved June 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2024-50349

National Vulnerability Database (NIST). (2025, 2 28). *NVD - CVE-2025-26466*. NVD. Retrieved June 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2025-26466

National Vulnerability Database (NIST). (2025, 5 21). *CVE-2025-48416*. Retrieved 6 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2025-48416

National Vulnerability Database (NVD) NIST. (2023, 1 17). *CVE-2022-37436*. CVE-2022-37436 Detail. https://nvd.nist.gov/vuln/detail/cve-2022-37436

National Vulnerability Database (NVD) NIST. (2023, 10 23). *CVE-2023-43622 Detail*. Retrieved 5 31, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2023-43622

National Vulnerability Database (NVD) NIST. (2024, April 4). *NVD - cve-2023-38709*. NVD. Retrieved May 31, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2023-38709

National Vulnerability Database (NVD) NIST. (2026, 5 10). CVE-2026-7259 Detail. https://nvd.nist.gov/vuln/detail/CVE-2026-7259

National Vulnerability Database (NVD) NIST. (2026, 5 10). *CVE-2026-7568 Detail*. Retrieved 5 31, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2026-7568

National Vulneravility Databae (NIST). (2025, 7 8). *CVE-2025-48384*. CVE-2025-48384. Retrieved 6 9, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2025-48384

NIST. (2023, 10 23). *CVE-2023-43622 Detail*. https://nvd.nist.gov/vuln/detail/CVE-2023-43622

NVD. (2023, 10 17). *CVE-2023-22084*. CVE-2023-22084. Retrieved 5 31, 2026, from https://nvd.nist.gov/vuln/detail/cve-2023-22084

NVD - CVE-2026-7261. (2026, 5 10). NVD. Retrieved May 31, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2026-7261

NVD - CVE-2026-7262. (2026, 5 10). NVD. Retrieved May 31, 2026, from https://nvd.nist.gov/vuln/detail/CVE-2026-7262

Smith, B. (2026, February 4). *What's the Cost of a Server for Small Business in 2026?* ServerMania. Retrieved June 9, 2026, from https://www.servermania.com/kb/articles/how-much-does-a-server-cost-for-a-small-business

VirtualBox. (n.d.). *Installing VirtualBox*. Oracle VirtualBox. Retrieved June 9, 2026, from https://www.virtualbox.org/manual/topics/installation.html#installation

VMWare. (n.d.). *What is Server Virtualization?* VMware. Retrieved June 9, 2026, from https://www.vmware.com/topics/server-virtualization